Probing Secure Composability Without Fresh Randomness: Theory and Application to Ascon

Vahid Jahandideh, Bart Mennink and Lejla Batina

Radboud University, Nijmegen, The Netherlands
{v.jahandideh,b.mennink,lejla}@cs.ru.nl

Abstract. Side-channel attacks (SCAs) pose a significant threat to the implementations of lightweight ciphers, particularly in resource-constrained environments where masking—the primary countermeasure—is constrained by tight resource limitations. This makes it crucial to reduce the resource and randomness requirements of masking schemes. In this work, we investigate an approach to minimize the randomness complexity of masking algorithms. Specifically, we explore the theoretical foundations of higher-order masking schemes that eliminate the need for *online* (fresh) randomness by relying solely on *offline* randomness present in the initial input shares.
We demonstrate that round-based ciphers with linear diffusion layers can support such *deterministic composition*, where the diffusion layer acts as a refresh subcircuit. This ensures that, up to a threshold number, probes placed across rounds remain independent. Based on this observation, we propose composition theorems for probing-secure masking. On the practical side, we instantiate our framework using known deterministic first- and second-order masked S-boxes and provide software implementations of Ascon's protected permutation.

Keywords: Side-Channel, Masking, Randomness Complexity, Ascon.

1 Introduction

Side-Channel Attacks and Masking. Side-channel leakages and the attacks exploiting them pose a significant threat to secure cryptographic implementations. A widely used countermeasure to mitigate these attacks is *masking*, which operates in two phases. First, inputs to the cipher (such as the nonce and key) are secret-shared into n shares, using *offline randomness*. Second, the cipher's gates (e.g., AND and XOR) are replaced with specialized mini-circuits called gadgets, which process n-shared inputs to produce n-shared outputs. These gadgets typically require fresh randomness—known as *online randomness*—for secure operation.

Generating high-quality randomness—particularly in the presence of side-channel leakages—is challenging and resource-intensive [CGZ20, CMM+24]. As a result, considerable research has focused on reducing the amount of randomness that masking requires [BBP+16, FPS17, SM21].

Lightweight ciphers, such as the newly standardized Ascon [NIS24], are explicitly designed for efficient performance in resource-constrained environments, which further motivates minimizing the complexity of protections. In this work, we explore a novel approach to reducing randomness consumption. Specifically, we investigate the feasibility of eliminating the reliance on online randomness altogether—a paradigm we call *deterministic masking*.

Deterministic Masking. Proposals for probing-secure deterministic gadgets already exist in the literature [NRR06, PAB+23, DDE+20, SM21]. In this work, we aim to extend this

concept by investigating the feasibility of achieving probing security at the circuit level. In general, this goal is challenging: even if each gadget is probing-secure in isolation, their composition can become insecure without *refresh gadgets* [Rep15, CPRR14].

The standard approach to address composition problem is to deploy refresh gadgets [BBD+16]. However, refresh gadgets require online randomness, which contradicts the principles of deterministic masking. To overcome this challenge, we show that the architecture of certain permutation-based ciphers—such as Ascon—inherently supports deterministic masking. Specifically, the diffusion layer at the end of each round effectively substitutes for refresh gadgets, enabling secure composition without the need for online randomness.

Bricklayer Design of Permutations. A public (i.e., key-less) cryptographic permutation serves as the core building block for various primitives. Notable examples include Keccak [BDPV13] used in SHA-3 [SHA15], Ascon-p used in Ascon and Isap modes [DEM+20], and Xoodoo used in Xoodyak mode [DHP+20].

At a high level, a permutation is typically composed of multiple rounds, each consisting of two main components: a set of non-linear operations (the *confusion layer*) and a set of linear bit-mixing operations (the *diffusion layer*). The non-linear operations are often implemented using smaller, identical S-box functions that operate in parallel. Daemen and Rijmen [DR20] use the term *bricklayer* to describe this architectural style. Because these permutations are key-less, there is no notion of round-key addition, and in general the diffusion layer is more complex than a simple bit-permutation.

Figure 1 illustrates a deterministic masking scheme for such a bricklayer design. In this figure, 𝕊S-box denotes the protected (i.e., masked) implementation of each S-box, while 𝕊Diffusion represents the masked implementation of the diffusion layer.

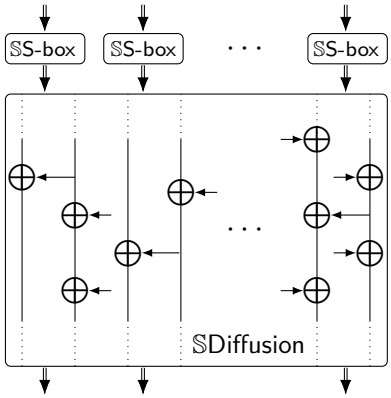


Figure 1: Deterministic masking for the bricklayer architecture. Notably, in the absence of refresh gadgets, 𝕊Diffusion linearly combines the output shares before the start of the next round.

Related Works. A commonly referenced software-oriented masking scheme for Ascon[1] is deterministic. However, its security has only been verified for a single round [GMSP23, DDE+20], leaving it unclear whether this scheme can maintain its security order when applied to multiple rounds.

At the gadget level, Nikova et al. [NRR06] introduced the Threshold Implementation (TI) approach, which requires more shares but operates deterministically. Their work

[1] https://github.com/ascon/simpleserial-ascon

also presented a first-order probing-secure deterministic multiplication gadget with $n = 4$ shares. Building on this foundation, several deterministic masking schemes for S-boxes have been proposed [BNN+12, MPL+11, PAB+23, SM21]. Bilgin et al. [BGN+14] extended the TI approach to higher-order security; however, at higher orders, TI-based designs typically require some randomness [DBR+15].

At the circuit level, Reparaz [Rep15] demonstrated that gate-level TI does not necessarily guarantee circuit-level security. For higher-order TI, Reparaz et al. [RBN+15] proposed adding a refresh layer to separate rounds, although Moos et al. [MMSS19] showed that this can be problematic without a supporting security proof. Moreover, inserting a refresh layer inherently introduces randomness, rendering the masking approach nondeterministic.

In pursuit of higher-order, circuit-level deterministic masking, Beyne et al. [BDZ20] proposed using linear cryptanalysis—specifically, correlation matrices—to bound the correlation between two probes in different rounds. A low correlation implies that probes are (approximately) independent and uniformly distributed. This approach was further developed in [BDMS22]. In contrast, our work also leverages cryptographic structure, but focuses more narrowly on the role of the diffusion layer. Our method is conceptually simpler and avoids correlation analysis. Additionally, the approach in [BDZ20, BDMS22] is limited to bounding leakage from two probes and relies on assumptions regarding independence of round transformations due to key additions. In Ascon, which we study in this work, these transformations are public and do not involve round keys.

1.1 Our Contribution

This work advances both the theoretical foundations and practical deployment of deterministic masking schemes. Our main contributions are as follows:

- **Diffusion as Implicit Refresh:** We show that in bricklayer cipher structures, the diffusion layer can effectively act as a refresh mechanism by combining the offline randomness embedded within each gadget. This hinders adversaries from exploiting probes across different rounds, thus enabling secure composition without explicit refresh gadgets.

- **Bridging the Gap with Simulation-Based Models:** We highlight that simulation-based frameworks and probe-propagation techniques [ISW03, BBD+16, CS20] inherently rely on fresh randomness, making them inadequate for verifying deterministic masking. To address this, we propose concrete refinements that allow applying simulation-based reasoning in a deterministic context.

- **Composability at First Order:** We formally characterize the conditions under which first-order secure gadgets compose securely. This includes a precise definition of independence for n-shared values, without requiring assumptions about the underlying secrets. We apply this result to round-based cipher constructions.

- **Higher-Order Security via Diffusion:** We show that probing the inputs and outputs of a diffusion layer preserves independence up to the layer's *linear branch number* [DR20]. This enables a provable composition of two rounds at higher orders. For more than two rounds, we provide a heuristic argument based on cryptographic properties of diffusion.

- **Implementation and Case Study on Ascon:** We implement our approach[2] on the NIST-standardized cipher Ascon [NIS24], demonstrating both first- and second-order probing security under deterministic masking. The S-box gadgets used are based on [SM21]. For the second-order case, we observe a structural separation: while

[2] https://github.com/vahid-jahandideh/deterministic-masking-ascon.git

the native secrets evolve non-linearly across rounds, the shares undergo only linear transformations. We use this to provide a probabilistic argument for second-order probing security across multiple rounds.

Outline. Section 2 provides the necessary mathematical background. Section 3 discusses additional preliminaries. In Section 4, we examine the limitations of simulation-based approaches and introduce our proposed modifications to support deterministic masking. Section 5 presents our main results for first-order probing security, which we extend to higher orders in Section 6, both within a deterministic masking framework. Next, Section 7 offers a case study applying this approach to Ascon. Finally, we conclude in Section 8.

2 Mathematical Background

In this section, we review the mathematical foundations necessary for verifying probing security. We discuss properties of functions such as bijectivity and explore the linear independence of random variables.

Notation. In this paper, we represent random variables, secrets, and intermediate values with capital letters (e.g., X), while their specific realizations or values are denoted using lowercase letters (e.g., x). Lists of shares and matrices are highlighted with bold letters (e.g., $\mathbf{X}$). The cardinality of a set $\mathcal{X}$ is represented as $|\mathcal{X}|$. The probability distribution of a random variable X is denoted by $\Pr(X)$, and $\Pr(X = x)$ specifies the probability that X takes the value x. Shares of a secret X are indexed as X_i, where the index i starts from one. To indicate that a gate G or circuit C is protected, we append the symbol $\mathbb{S}$ to its name (e.g., $\mathbb{S}\mathsf{G}$ and $\mathbb{S}\mathsf{C}$).

Probability Computation from LUT. Security evaluation of protected circuits often requires the computation of (conditional) probabilities. Here, we describe how such probabilities can be precisely obtained using a lookup table (LUT) representation.

Let random variables X and Y be connected through a deterministic function $Y = F(X, R)$, where R is an r-bit random variable, and X is uniformly distributed over $|X|$ values. A LUT with $2^r|X|$ rows represents all possible values taken by X and Y. Additional columns can be appended to this LUT to compute values of a function $G(X, Y)$. Using the LUT, the probability $\Pr(G_1(X, Y) = g_1)$ is computed as:

$$\Pr(G_1(X, Y) = g_1) = \frac{\#\{\text{rows} \mid G_1(X, Y) = g_1\}}{2^r|X|},$$

and the conditional probability $\Pr(G_1(X, Y) \mid G_2(X, Y))$ is determined by:

$$\Pr(G_1(X, Y) = g_1 \mid G_2(X, Y) = g_2) = \frac{\#\{\text{rows} \mid G_1(X, Y) = g_1 \text{ and } G_2(X, Y) = g_2\}}{\#\{\text{rows} \mid G_2(X, Y) = g_2\}}.$$

Definition 1 (Joint Independence)**.** A set of random variables $\{X_1, \ldots, X_l\}$ is said to be m-jointly independent ($m \leq l$) if, for any subset of size m, $\{X_{i_1}, \ldots, X_{i_m}\}$, the joint probability satisfies:

$$\Pr(X_{i_1}, \ldots, X_{i_m}) = \Pr(X_{i_1}) \cdots \Pr(X_{i_m}).$$

For example, given random binary variables R_1 and R_2, the set $\{R_1, R_2, R_1 \oplus R_2\}$ is 2-jointly independent.

Bijective Mapping and Collisions. A function (also referred to as a mapping) $F\colon \{0,1\}^m \to \{0,1\}^m$ is *bijective* if it establishes a *one-to-one correspondence* between elements of its *domain* and *codomain*. A bijective F is also known as a *permutation* and is invertible. Conversely, if F is not bijective, there exist distinct inputs x_1 and x_2 in its domain such that $F(x_1) = F(x_2)$. In this case, we say that F has a *collision*.

Polynomials and Parity Relations. Given a set of binary and independent random variables $\{X_1, \ldots, X_l\}$, we define binary polynomials as:

$$F_j = \bigoplus_{i=1}^{l} c_i^j X_i,$$

where the constants c_i^j belong to $\{0,1\}$. The Hamming weight (HW) of a polynomial F_j is defined as $\mathsf{wt}(F_j) = \sum_i c_i^j \in \mathbb{N}$.

In a set of polynomials $\mathcal{F} = \{F_1, \ldots, F_m\}$, we are interested in identifying *parity relations*, and, in particular, the one with the smallest HW. A parity relation is a subset of $\mathcal{F}$ that sums to zero:

$$\bigoplus_{j=1}^{m} d_j F_j = 0,$$

where its HW is given by $\sum_j d_j$ with d_j belonging to $\{0,1\}$. If no such parity relation exists, the m polynomials are said to be *linearly independent*. Since all relations among the polynomials are linear, we conclude that these polynomials (random variables) are m-jointly independent.

For example, let $l = 3$ and $\mathcal{F} = \{X_1 \oplus X_2, X_1 \oplus X_3, X_2 \oplus X_3, X_1 \oplus X_2 \oplus X_3, X_1\}$. In $\mathcal{F}$, there is no parity relation with Hamming weight less than 3; hence, the polynomials are 2-jointly independent.

3 Preliminaries

Circuit View. The computations of a cipher are typically represented by a circuit C, viewed as an *acyclic* and *directed* graph with vertices $\mathcal{V}$ and edges $\mathcal{E}$. Each vertex in $\mathcal{V}$ corresponds to an *atomic* operation (e.g., XOR or AND), while the edges in $\mathcal{E}$ represent the operands passed between these operations. In hardware terminology, such operations are called *gates*, and their operands are *wires*. In this paper, the terms "operation" and "gate," as well as "operand" and "wire," will be used interchangeably.

3.1 Our Security Model

Following common practice in the field, we assume that the side-channel adversary has full knowledge of both the unprotected circuit C and the protected circuit $\mathbb{S}\mathsf{C}$. The adversary's goal is to use side-channel leakage to learn the secret values carried on the wires.

Probing Adversary. In the probing model introduced by Ishai et al. [ISW03], an adversary can place up to d probes on a running circuit to learn the values of the corresponding wires. A protected circuit $\mathbb{S}\mathsf{C}$ is said to be secure in this model if an adversary with d probes on $\mathbb{S}\mathsf{C}$ cannot gain any information about the secret variables processed by the corresponding C.

Extended Probing. To account for practical physical effects, the standard probing model has been extended to capture phenomena such as *glitches* and *transition* leakages [FGMDP+18, CS21]. Glitch leakage can often be mitigated by introducing *synchronization* layers, while transition leakage can be addressed through resetting or re-initializing *registers*—neither of which requires fresh randomness. Additional concerns include *micro-architectural* [GD23a] and *coupling* leakages [DBG+17], which may be mitigated by adopting higher-order masking or applying device-specific countermeasures. We will return to these practical aspects in Section 7.

Our initial goal is to establish the *feasibility* of higher-order deterministic masking. For the theoretical discussions in this work, we operate under the standard probing model. Following Ishai et al. [ISW03], we assume that the adversary cannot probe the circuit during the initial encoding or final decoding of secrets and shares.

3.2 Boolean Masking

To protect C from side-channel leakage, each variable in $\mathcal{E}$ is secret-shared via a technique commonly called *masking*. For a standalone u-bit variable $X \in \mathbb{F}_{2^u}$, masking represents X as $\mathbf{X} = \{X_1, \ldots, X_n\}$, a set of n shares randomly chosen from $\mathbb{F}_{2^u}$ such that $\oplus_{i=1}^{n} X_i = X$. We call $\mathbf{X}$ an n-sharing and X the *native* or secret variable. In an n-sharing, any subset of $(n-1)$ shares is indistinguishable from $(n-1)$ independent random elements of $\mathbb{F}_{2^u}$, even if X is known.

Masking a circuit C proceeds by encoding its inputs and replacing its gates (the vertices in $\mathcal{V}$) with *gadgets*. A gadget replicates the function of a gate while accepting n-shared inputs and producing n-shared outputs.

For *additive-affine* operations,[3] the corresponding gadget design is straightforward. For example, if $\mathsf{G}\colon \mathbb{F}_{2^u} \to \mathbb{F}_{2^u}$ is a single-input single-output function, then the gadget $\mathbb{S}\mathsf{G}\colon (\mathbb{F}_{2^u})^n \to (\mathbb{F}_{2^u})^n$ takes an n-sharing $\mathbf{X}$ and outputs an n-sharing $\mathbf{Y}$ by setting $Y_1 = \mathsf{G}(X_1)$ and $Y_i = \mathsf{G}(X_i) \oplus \mathsf{G}(0)$ for $2 \le i \le n$. For non-linear operations (e.g., $\mathsf{S\text{-}box}$), gadgets tend to be more complex and case-specific.

Although composing gadgets in a *trivial* way yields a functionally correct circuit, it can introduce security flaws. To address these, *refresh gadgets* are often inserted. A refresh gadget takes an n-sharing $\mathbf{X}$ and, using *online randomness*, produces a new n-sharing $\mathbf{Y}$ of the same secret X such that $\mathbf{X}$ and $\mathbf{Y}$, given X, are independent in distribution. In other words, $\mathbf{X}$ and $\mathbf{Y}$ are two independent n-sharings.

Deterministic Masking. We use the term *deterministic masking* to refer to any gadget or protected circuit ($\mathbb{S}\mathsf{C}$) that operates *without* consuming online (fresh) randomness. Early work by Nikova et al. [NRR06] identified a first-order probing-secure deterministic multiplication gadget with $n = 4$ shares, establishing the concept of Threshold Implementation (TI) for gadget-level protection. Many subsequent works have adopted TI-based techniques, which often yield deterministic masking.

However, extending from a deterministic gadget to a fully deterministic masked circuit is more challenging. Primarily because the *probe propagation* framework (see Section 4), which is a key approach for proving the security of compositions, relies on the use of online randomness, particularly through refresh gadgets. The main contribution of this paper is to lay the groundwork for an alternative composition strategy that achieves probing security *without* the need for online randomness.

[3] An operation G is additive-affine if, for all x and y in its domain, $\mathsf{G}(x+y) = \mathsf{G}(x) + \mathsf{G}(y) - \mathsf{G}(0)$.

3.3 Threshold Implementation

The Threshold Implementation (TI) approach for masking non-linear functions relies on three core properties: *correctness*, *non-completeness*, and *uniformity*. TI is well-regarded for its effectiveness in mitigating information leakage due to hardware glitches; however, it typically requires more shares than other masking methods in the probing model.

Properties of TI [BGN+14, NRR06]. Let $F\colon \mathbb{F}_{2^u}^{m_{\text{in}}} \to \mathbb{F}_{2^u}^{m_{\text{out}}}$ be a function defined by

$$F(X^1, \ldots, X^{m_{\text{in}}}) = (Y^1, \ldots, Y^{m_{\text{out}}}),$$

and let $\mathbb{S}F\colon \mathbb{F}_{2^u}^{m_{\text{in}}n} \to \mathbb{F}_{2^u}^{m_{\text{out}}n}$ be an n-share masked implementation of F. Let the input sharings be denoted as $\mathbf{X}^i = \{X_1^i, \ldots, X_n^i\}$ for $1 \le i \le m_{\text{in}}$, and the output sharings as $\mathbf{Y}^i = \{Y_1^i, \ldots, Y_n^i\}$ for $1 \le i \le m_{\text{out}}$. The implementation $\mathbb{S}F$ is a Threshold Implementation (TI) of F at order d if it satisfies the following properties:

- *Correctness.* For all inputs, the recombination of the output shares matches the function output on the recombined inputs:

$$F\big(\oplus_{j=1}^n X_j^1, \ldots, \oplus_{j=1}^n X_j^{m_{\text{in}}}\big) = \big(\oplus_{j=1}^n Y_j^1, \ldots, \oplus_{j=1}^n Y_j^{m_{\text{out}}}\big).$$

- *Non-completeness.* Any set of up to d output shares Y_j^i depends on at most $n-1$ shares from each input sharing $\mathbf{X}^i$.

- *Uniformity.* For any input $(X^1, \ldots, X^{m_{\text{in}}})$, the output $\mathbb{S}F$ must produce a uniform n-sharing of $F(X^1, \ldots, X^{m_{\text{in}}})$. That is, each output $\mathbf{Y}^i$ should form an n-sharing where any subset of $n-1$ shares Y_j^i $(1 \le j \le n)$ is uniformly distributed over $\mathbb{F}_{2^u}^{n-1}$, regardless of the distribution of the underlying secrets.

As a concrete example, Nikova et al. [NRR06] proposed a first-order (i.e., $d = 1$) TI gadget for the binary (i.e., $u = 1$) AND function. Given two secret variables A and B with 4-share encodings $\mathbf{A} = \{A_1, \ldots, A_4\}$ and $\mathbf{B} = \{B_1, \ldots, B_4\}$, the gadget computes $\mathbf{C} = \{C_1, \ldots, C_4\}$ such that $C = A \cdot B$, with:

$$\begin{cases} C_1 = (1 \oplus A_3 \oplus A_4)(1 \oplus B_2 \oplus B_3) \oplus B_4 \oplus A_2, \\ C_2 = (1 \oplus A_1 \oplus A_3)(1 \oplus B_1 \oplus B_4) \oplus B_3 \oplus A_4, \\ C_3 = (A_2 \oplus A_4)(B_1 \oplus B_4) \oplus B_2 \oplus A_2, \\ C_4 = (A_1 \oplus A_2)(B_2 \oplus B_3) \oplus B_1 \oplus A_1. \end{cases}$$

This construction satisfies the TI conditions:

- Correctness: $\oplus_{i=1}^4 C_i = (\oplus_{i=1}^4 A_i) \cdot (\oplus_{i=1}^4 B_i)$ for all $2^4 \times 2^4$ input combinations.

- Non-completeness: Each output share C_i depends on at most $n - 1 = 3$ shares from both $\mathbf{A}$ and $\mathbf{B}$.

- Uniformity: For any secret values $A = a$, $B = b$, the output $\mathbf{C}$ is a uniform 4-sharing of ab, even though ab may be biased. That is, all 4-tuples (C_1, C_2, C_3, C_4) summing to ab are equally likely.

Recently, Jahandideh et al. [JSB25] extended this deterministic multiplication gadget to higher security orders ($d > 1$) and to larger fields ($u > 1$).

Limitations of Circuit-Level TI. As discussed in Section 1, extending Threshold Implementations to the circuit level at higher orders ($d > 1$) remains a significant challenge. Existing approaches, such as "Consolidating Masking Schemes" [RBN+15], apply TI gadgets locally and rely on additional refresh gadgets to enable circuit-level composition. However, Moos et al. [MMSS19] demonstrated through several concrete attacks that ad hoc refresh strategies do not reliably ensure higher-order security. They advocate instead for the proper use of standard composition techniques and probe-propagation-based frameworks.

Unfortunately, both refreshing mechanisms and standard probe-propagation techniques inherently require online randomness. As a result, these methods are fundamentally incompatible with deterministic masking, where no fresh randomness is available during execution.

3.4 Composition Problem

Protecting a circuit requires more consideration than simply designing protected gates and combining them. It is a well-known issue that trivially composing d-probing secure gadgets can leave them vulnerable to fewer than d probes. This concern has been addressed in several works [CPRR14, Rep15, BBD+16]. However, since the composition challenge is central to our discussion, we pause to clarify it further with an example.

Example 1. Suppose a 3-bit $\mathsf{S\text{-}box}$ is formed by combining χ_3 and a linear layer. Given input bits $\{A, B, C\}$, χ_3 computes the output bits as:

$$\begin{cases} A' = A \oplus (1 \oplus B)C, \\ B' = B \oplus (1 \oplus C)A, \\ C' = C \oplus (1 \oplus A)B. \end{cases}$$

The subsequent linear layer computes $A'' = A' \oplus B'$, $B'' = B'$, and $C'' = 1 \oplus C'$. For this $\mathsf{S\text{-}box}$, we propose the following $\mathbb{S}\mathsf{S\text{-}box}$ at $n = 2$, where $\mathbb{S}\chi_3$ operates as:

$$\begin{cases} A'_1 = [A_1 \oplus (1 \oplus B_1)C_1] \oplus B_2C_1, & A'_2 = [A_2 \oplus (1 \oplus B_1)C_2] \oplus B_2C_2, \\ B'_1 = [B_1 \oplus (1 \oplus C_1)A_1] \oplus C_2A_1, & B'_2 = [B_2 \oplus (1 \oplus C_1)A_2] \oplus C_2A_2, \\ C'_1 = [C_1 \oplus (1 \oplus A_1)B_1] \oplus A_2B_1, & C'_2 = [C_2 \oplus (1 \oplus A_1)B_2] \oplus A_2B_2. \end{cases} \tag{1}$$

The masked linear layer works as:

$$A''_1 = A'_1 \oplus B'_1, \quad A''_2 = A'_2 \oplus B'_2, \quad B''_1 = B'_1, \quad B''_2 = B'_2, \quad C''_1 = 1 \oplus C'_1, \quad C''_2 = C'_2.$$

This $\mathbb{S}\mathsf{S\text{-}box}$ is first-order probing secure. The security claim can be directly proved by showing that each intermediate V is independent of the secrets (A, B, C). That is, $\Pr(V \mid A, B, C) = \Pr(V)$. Verifying this involves enumerating all possible inputs, yielding $2^{3n} = 64$ inputs in this example.[4]

Now, consider a 3-bit permutation $\mathsf{f}^{(r)}$, constructed by applying $\mathsf{S\text{-}box}$ r times, i.e.,

$$\mathsf{f}^{(r)}(A, B, C) = \mathsf{S\text{-}box}(\ldots(\mathsf{S\text{-}box}(A, B, C))\ldots).$$

We show that trivially composing the given $\mathbb{S}\mathsf{S\text{-}box}$ does not result in a secure computation of $\mathbb{S}\mathsf{f}^{(r)}(A_1, A_2, B_1, B_2, C_1, C_2)$ for $r > 1$. For example, by enumerating all input shares, for A''_1 at the output of $\mathbb{S}\mathsf{f}^{(16)}$, we find:

$$\Pr(A''_1 = 0 \mid A = 0, B = 0, C = 1) = 0, \quad \text{whereas} \quad \Pr(A''_1 = 0) = \frac{1}{2}.$$

[4] An intuitive explanation for the security is that each intermediate is *blinded* by separate shares of the inputs, as elaborated in the next section.

This shows that A_1'' is *dependent* on native variables, violating first-order probing security.

Although this S-box is bijective, the given $\mathbb{S}$S-box is *not* a bijective gadget and has *collisions*. For instance, the inputs:

$$\begin{cases} (A_1 = 1, A_2 = 1, B_1 = 1, B_2 = 1, C_1 = 0, C_2 = 0)\,, \\ (A_1 = 0, A_2 = 0, B_1 = 0, B_2 = 0, C_1 = 1, C_2 = 1)\,, \end{cases}$$

map to the same output. Collisions reduce the output space, consume initial randomness, and make the intermediates of subsequent rounds secret-dependent. For $\mathsf{f}^{(r)}$, collisions increase with r, as shown in Figure 2.

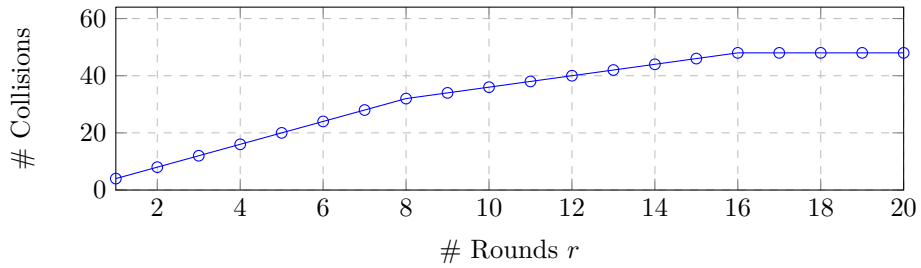


Figure 2: More collisions occur with increasing number of rounds.

In the following, we briefly explain the fundamental cause of the security loss in this example:

The probing security of a gadget, such as a $\mathbb{S}$S-box, is typically proven under the assumption that its inputs are both independent and uniformly distributed. However, if a collision occurs, the gadget's output distribution will no longer remain uniform. In this circuit, the output of one gadget is fed into the next. As soon as the inputs to any gadget deviate from uniformity, its probing security can no longer be guaranteed. □

Role of Online Randomness. A generic solution to composition issues involves incorporating online randomness, particularly through the insertion of refresh gadgets. Online randomness compensates for entropy loss caused by collisions and blocks the *propagation* of adversary probes (detailed in the following section).

4 Simulation-Based Security and its Limitations

Direct verification of d-probing security, which involves checking the independence of any d-tuple of intermediates from native variables, becomes impractical as the sharing order n increases. The *simulation* approach by Ishai et al. [ISW03] addresses this challenge. To prove d-probing security, one shows that the distribution of up to d probes (on intermediates) can be *simulated*, i.e., reproduced in an indistinguishable manner, using fewer than n shares of each input.

The standard simulation approach follows basic rules. Let V_1 and V_2 be intermediates (not necessarily shares of a secret), and let R represent an online randomness variable:

- If a probe P is of the form $P = R \oplus V_1$, and R is not involved in the computation of any other probe, then R *hides* V_1. In this case, the distribution of P can be simulated using an independent random variable R' as $P = R'$, without requiring V_1 for the simulation.

- If a probe P is of the form $P = V_1 \oplus V_2$ or $P = V_1V_2$, both V_1 and V_2 are required for the simulation.

Definition 2 (Probe Propagation)**.** If simulating a probe P requires knowledge of some intermediate V, we say that P propagates to V, and V is called a *propagated probe*.

Probes always propagate toward inputs. The propagation rules are depicted in Figure 3. Notably, the addition of online randomness, as stated above, halts further propagation.

4.1 Composition Rules

The simulation rules are useful for proving the probing security of standalone gadgets. However, they are insufficient for verifying the security of compositions. Barthe et al. [BBD+16] introduced the concept of *strong non-interference* (SNI) to address this issue. A gadget is SNI if simulating up to a threshold number of output shares does not require any input shares. Thus, an SNI gadget acts as a barrier, stopping further propagation of probes. Cassiers and Standaert [CS20] later refined this concept by introducing *probe isolating non-interference* (PINI) gadgets. PINI gadgets are trivially composable and do not require additional refresh gadgets at interfaces for secure composition.

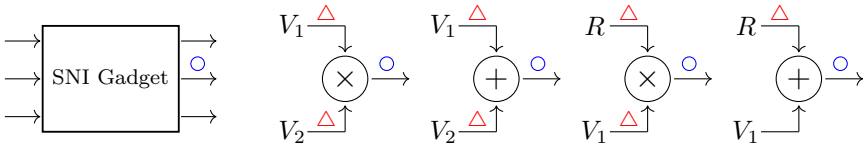


Figure 3: Probe propagation rules. Blue circles represent probes, and red triangles represent propagated probes. V_1 and V_2 are intermediates, and R is a randomness variable.

Simulation Needs Online Randomness. The probe elimination technique, as well as the design of SNI and PINI gadgets, inherently requires randomness.[5] This reliance on randomness limits their deployment for proving the probing security of deterministic masking schemes, where the use of online randomness is avoided. This paper explores tweaks to handle the absence of online randomness. As a first step, we demonstrate that simulation-based security is more restrictive than the core definition of probing security.

4.2 Limitations of Simulation-Based Security Proof

Given a set of probes $\mathbf{P}$ and a set of natives $\mathbf{V}$ (with any distribution), *probing security* requires that $\mathbf{P}$ conveys no information about $\mathbf{V}$, i.e.,

$$\Pr(\mathbf{P} \mid \mathbf{V}) = \Pr(\mathbf{P}).$$

Probing security addresses the equivocation about natives and, in some cases, is a broader concept than simulation of the probes. The following examples clarify this point.

A 2-input Case. At $n = 2$, let $\{A_1, A_2\}$ and $\{B_1, B_2\}$ be shares of secrets A and B. For simulating a probe P of the form $P = A_1 \oplus B_1B_2$, the introduced simulation rules require A_1, B_1, and B_2. Specifically, since both shares of B are required, the approach suggests that P depends on B. However, if the shares of A and B are independent, direct enumeration over all $2^{2n} = 16$ possible inputs reveals that P is independent of A and B. In this case, A_1 hides the secret-dependent term B_1B_2. □

[5] If no online randomness is involved, input shares *uniquely* determine each output share. Consequently, simulating (reproducing) output shares without using any of the input shares becomes impossible.

A 3-input Case. Another example involves probing intermediates of $\mathbb{S}_{\chi_3}$ as defined in Equation (1). For instance, a probe on A'_1 results in:

$$P = [A_1 \oplus (1 \oplus B_1)C_1] \oplus B_2C_1.$$

Simulating P (based on the standard rules) requires the shares B_1, B_2, C_1, and A_1. Specifically, both shares of B are required, indicating that the standard simulation approach cannot prove probing security. However, the randomness brought in by A_1 hides the secret-dependent term:

$$(1 \oplus B_1)C_1 \oplus B_2C_1 = (1 \oplus B)C_1. \qquad \square$$

These examples illustrate that probing security is a broader notion than the standard simulation-based security. They also demonstrate how randomness introduced by the shares can be used to blind probes. However, this approach has an important prerequisite:

the shares of inputs must be independent.

Efficiency vs. Generality. Barthe et al. [BBD+15b, Appendix E] also highlight the significant limitation of (standard) simulation-based approaches for probing security. On the one hand, simulation-based security enables more efficient verification by providing compositional rules that reduce probing security of the entire circuit to manageable subproblems concerning its constituent gadgets. On the other hand, as our preceding examples illustrate, this approach may fail to prove probing security for certain gadgets.

These limitations of simulation-based probing security are particularly relevant when selecting tools for the formal verification of masking. For instance, the current version of maskVerif [BBC+19], which is simulation-based, produces false positives for the examples in this subsection. Meanwhile, SILVER [KSM20], which directly checks the statistical independence of probes from native variables, does confirm the probing security of the same examples.

To address this issue, Meyer et al. [MBR19] proposed using mutual information. In this paper, we propose an alternative: we extend simulation rules by using the blinding effect of independent input shares. Our approach generalizes Coron's probe elimination technique [Cor18] to work in deterministic masking settings.

4.3 Probe Elimination

In a set of probes $\mathbf{P}$, let a probe P_i be of the form $P_i = R \oplus P'$, where P' is a function of intermediates other than R. If the randomness R is not involved in any other probe, then P_i can be *eliminated* from $\mathbf{P}$, and its distribution can be simulated using an independent randomness R'. In this case, R *hides* the intermediates in P'. We also say, R *blinds* the probe P_i. The probe elimination process can iteratively continue by updating $\mathbf{P} \leftarrow \mathbf{P} - P_i$. The remaining intermediates required to compute the final $\mathbf{P}$ are needed for the simulation. We clarify this approach with an example.

Example 2. Let the sharing order be $n = 2$, and let $\mathbf{A} = \{A_1, A_2\}$ be the shares of the native A. Assume the adversary has the following probes:

$$P_1 = R_1 \oplus A_1A_2 \oplus R_2, \quad P_2 = R_2 \oplus A_1 \oplus A_2, \quad P_3 = A_1,$$

where R_1 and R_2 are randomness variables. Applying the probe elimination approach:

- P_1 is of the form $P_1 = R_1 \oplus P'$, and since R_1 is not involved in P_2 and P_3, we eliminate P_1, updating $\mathbf{P} = \{P_2, P_3\}$.
- In the second iteration, $P_2 = R_2 \oplus P'$, and R_2 is not involved in the computation of the current probes. We eliminate P_2, updating $\mathbf{P} = \{P_3\}$.

The remaining probe $P_3 = A_1$ requires only one input share (A_1) for simulation. Thus, the adversary's probes can be simulated with knowledge of only one input share. $\square$

Proving Probing Security Using Probe Elimination. To prove probing security of a circuit $\mathbb{S}\mathsf{C}$, we can use the probe elimination technique. Consider any set of d probes on intermediate values, each having the following form:

$$\mathbf{P} = \{P_1 = R_1 \oplus P'_1,\ \ldots,\ P_d = R_d \oplus P'_d\},$$

where R_i, for $1 \leq i \leq d$, are randomness variables used only in the specific (explicitly shown) locations associated with each probe. Under these conditions, probe elimination ensures that all d probes can be "eliminated" after d iterations, thus proving security.

A slight modification generalizes the above scenario:

$$\mathbf{P} = \left\{P_1 = \left(\bigoplus_{i\in\mathcal{I}_1} R_i\right) \oplus P'_1,\ \ldots,\ P_d = \left(\bigoplus_{i\in\mathcal{I}_d} R_i\right) \oplus P'_d\right\},$$

with the assumption that each probe has a unique randomness variable used exclusively by that probe. Concretely, in each set $\mathcal{I}_{j^*}$, for $1 \leq j^* \leq d$, there is exactly one element that does not appear in

$$\bigcup_{j\neq j^*} \mathcal{I}_j.$$

Probe elimination applies in this setting and results in the full elimination of all probes.

Input Shares as the Source of Randomness. We extend this principle to deterministic masking. In the absence of online randomness, we treat the input shares—generated using offline randomness—as the source of blinding.

Suppose the circuit C processes a state $S = \{X^1, X^2, \ldots, X^b\}$. Each secret X^i is encoded into n shares $\mathbf{X}^i = \{X_{(n-1)i+1}, \ldots, X_{ni}\}$, consuming a total of $(n-1)b$ independent randomness variables. Let $\{X_1, X_2, \ldots, X_{nb}\}$ denote the full set of shares.

Now consider a probe of the form:

$$P = \left(\bigoplus_{i\in\mathcal{I}} X_i\right) \oplus F\left(S, \{X_i \mid i \in \mathcal{J}\}\right),$$

where $\mathcal{I} \cap \mathcal{J} = \emptyset$, and F is an arbitrary (possibly non-linear) function of the inputs.

If there exists an index $i \in \mathcal{I}$ such that not all n shares of the corresponding secret $X^{\lceil i/n \rceil}$ appear in $\mathcal{I} \cup \mathcal{J}$, then X_i blinds the probe P. In that case, the distribution of P becomes independent of the native secrets, and P can be safely eliminated.

This observation forms the basis of our analysis in Sections 6 and 7 for the case of $d > 1$, where we show that structural properties of the cipher ensure that such blinding holds across multiple compositions.

5 First-Order Deterministic Masking

Blinding using input shares and the non-completeness property (as discussed in Section 3.3) are effective techniques for achieving probing security at the level of individual gadgets. However, these techniques alone do not guarantee the security of *deterministic masking* when composed across an entire circuit.

In this section, we focus on the simpler case of *first-order* probing security. In the context of Threshold Implementations (TI), Dhooghe et al. [DNR19] demonstrated that if all layers in a circuit are non-complete and uniform, then the composition remains secure at first order.

Related ideas have also been considered in the context of fault attacks. Notably, Daemen et al. [DDE+20] argue that when each gadget (e.g., masked $\mathsf{S\text{-}box}$) is a permutation over

the shares and is applied to uniformly shared inputs, the uniformity is preserved across rounds—ensuring first-order security without fresh randomness. While this aligns with the spirit of our approach, we revisit the argument more critically. In particular, we will provide a counterexample showing that the permutation property alone is not sufficient for guaranteeing probing security.

Building on these observations, we now formalize the conditions under which a deterministic masked circuit preserves first-order probing security:

Lemma 1. *Let n be the sharing order. A masked circuit composed entirely of first-order probing-secure gadgets is itself first-order probing secure if each gadget with m input variables receives m-jointly independent n-sharings at its inputs.*

Proof. The adversary is allowed to place a single probe, which necessarily targets an intermediate value within one of the gadgets. Since each gadget is assumed to be first-order probing secure, the probed value is independent of the secrets—*provided* that the gadget's input sharings are m-jointly independent. Because this joint independence holds across the circuit, each gadget behaves as it would in isolation. Consequently, any single probe—regardless of placement—yields no information about the native secrets, ensuring first-order probing security of the entire circuit. □

5.1 Independence of n-Sharings

A key element of Lemma 1 is the (joint) independence of the input n-sharings that feed into a gadget. In this subsection, we formalize this notion of independence. Recall that an n-sharing of a native variable X is denoted by $\mathbf{X} = \{X_1, \ldots, X_n\}$, where $\mathbf{X}$ is uniformly distributed over all assignments satisfying $\bigoplus_{i=1}^{n} X_i = X$. As a result, any subset of $(n-1)$ shares in $\mathbf{X}$ also follows a uniform distribution. Note that this definition places no constraints on the distribution of X: it might be public (e.g., a nonce), fixed (e.g., an initialization vector), or secret. Similarly, the definition of independence for n-sharings does not depend on whether their underlying natives are secret or public.

More concretely, for an n-sharing $\mathbf{X}$,

$$\Pr\big(\mathbf{X} = \mathbf{x} \mid X = x\big) = \begin{cases} 0, & \text{if } \bigoplus_{i=1}^{n} x_i \neq x, \\ \dfrac{1}{2^{(n-1)u}}, & \text{otherwise,} \end{cases}$$

where u is the bit-length of the underlying field $\mathbb{F}_{2^u}$.

Two n-sharings $\mathbf{X}$ and $\mathbf{Y}$, corresponding to natives X and Y, are said to be *independent* if

$$\Pr\big((\mathbf{X}, \mathbf{Y}) \;\big|\; (X, Y)\big) \;=\; \Pr(\mathbf{X} \mid X)\;\Pr(\mathbf{Y} \mid Y).$$

This notion naturally extends to sets of more than two n-sharings:

Definition 3 (Independence of n-sharings)**.** A collection of n-sharings $\{\mathbf{X}_1, \ldots, \mathbf{X}_l\}$ is *independent* if

$$\Pr\Big((\mathbf{X}_1, \ldots, \mathbf{X}_l) \;\Big|\; (X_1, \ldots, X_l)\Big) \;=\; \prod_{i=1}^{l} \Pr\big(\mathbf{X}_i \mid X_i\big).$$

Equivalently, for any valid assignment of these n-sharings, i.e., $\{\mathbf{x}_1, \ldots, \mathbf{x}_l\}$ summing to $\{x_1, \ldots, x_l\}$ respectively, the probability is $\frac{1}{2^{(n-1)ul}}$, and 0 otherwise.

For example, the input and output of a refresh gadget form two independent n-sharings.[6] Moreover, cascading a refresh gadget $l-1$ times yields l independent n-sharings.

[6] In the linear refresh gadget case, the only connection is the trivial relation $\bigoplus_{i=1}^{n} X_i = \bigoplus_{i=1}^{n} Y_i$.

It may happen that a set of l n-sharings is not fully independent, yet every subset of size $m \leq l$ is jointly independent. In this scenario, we say the n-sharings are *m-jointly independent*.

Definition 4 (m-Joint Independence of n-sharings)**.** A collection of n-sharings $\{\mathbf{X}_1, \ldots, \mathbf{X}_l\}$ is *m-jointly independent* if for any subset of indices $i_1, \ldots, i_m$,

$$\Pr\Big((\mathbf{X}_{i_1}, \ldots, \mathbf{X}_{i_m}) \;\Big|\; (X_{i_1}, \ldots, X_{i_m})\Big) \;=\; \prod_{j=1}^{m} \Pr\big(\mathbf{X}_{i_j} \mid X_{i_j}\big).$$

As an illustration, suppose $\mathbf{X}_1$ and $\mathbf{X}_2$ are independently sampled n-sharings, and let $\mathbf{X}_3 = \mathbf{X}_1 \oplus \mathbf{X}_2$, where $\oplus$ is applied share-wise. Then $\{\mathbf{X}_1, \mathbf{X}_2, \mathbf{X}_3\}$ is 2-jointly independent.

The connection between a set of (m-jointly) independent n-sharings $\{\mathbf{X}_1, \ldots, \mathbf{X}_l\}$ and their respective natives can be "broken" by discarding one share. Specifically, if $\mathbf{X}^o \subset \mathbf{X}$ contains only $(n-1)$ shares, then $\Pr(\mathbf{X}^o \mid X) = \Pr(\mathbf{X}^o)$. From this observation, one can equivalently define a set of n-sharings $\{\mathbf{X}_1, \ldots, \mathbf{X}_l\}$ as independent if

$$\Pr\big(\mathbf{X}_1^o, \ldots, \mathbf{X}_l^o\big) \;=\; \prod_{i=1}^{l} \Pr\big(\mathbf{X}_i^o\big),$$

where each $\mathbf{X}_i^o \subset \mathbf{X}_i$ contains $(n-1)$ of the n shares in $\mathbf{X}_i$.

We next show that a masked version of any linear, bijective function preserves the independence of its input n-sharings:

Lemma 2. *Let F be a bijective function, and let $\mathbb{S}F$ be its masked counterpart. If the input to $\mathbb{S}F$ consists of l independent n-sharings $\{\mathbf{X}_1, \ldots, \mathbf{X}_l\}$, then the output $\big(\mathbf{Y}_1, \ldots, \mathbf{Y}_l\big) = \mathbb{S}F(\mathbf{X}_1, \ldots, \mathbf{X}_l)$ is also a set of independent n-sharings.*

Proof. For any assignment of output shares $\{\mathbf{y}_1, \ldots, \mathbf{y}_l\}$ with corresponding natives $\{y_1, \ldots, y_l\}$, there is a unique preimage $\{\mathbf{x}_1, \ldots, \mathbf{x}_l\} = \mathbb{S}F^{-1}(\mathbf{y}_1, \ldots, \mathbf{y}_l)$ with natives $\{x_1, \ldots, x_l\} = F^{-1}(y_1, \ldots, y_l)$. Hence,

$$\begin{aligned}\Pr\big((\mathbf{Y}_1 = \mathbf{y}_1, \ldots, \mathbf{Y}_l = \mathbf{y}_l) \;\big|\; (Y_1 = y_1, \ldots, Y_l = y_l)\big) \\ &= \Pr\big((\mathbf{X}_1 = \mathbf{x}_1, \ldots, \mathbf{X}_l = \mathbf{x}_l) \;\big|\; (X_1 = x_1, \ldots, X_l = x_l)\big) \\ &= \tfrac{1}{2^{(n-1)u\,l}},\end{aligned}$$

which, by Definition 3, confirms the independence of $\{\mathbf{Y}_1, \ldots, \mathbf{Y}_l\}$. □

Bijectivity vs. n-Sharing. In our deterministic masking setup, a gadget's bijectivity is crucial for preserving the randomness of its inputs. However, bijectivity alone does not ensure that an n-shared input will yield an n-shared output. The following example illustrates why these two notions differ.

Example 3. Consider a gadget $\mathbb{S}\mathsf{G}_i$ operating on an n-shared input $\mathbf{X}$. Suppose it produces the output

$$\mathbf{Y} \;=\; \{\, X_1 \oplus X_i,\; X_2,\; \ldots,\; X_n\},$$

i.e., it modifies share X_1 by adding share X_i. Although this gadget is bijective—one can uniquely recover all input shares from the output shares—it does not preserve the n-sharing property. Hence, it compromises compositional security. Specifically, observe that

$$\mathbf{Y} \;=\; \mathbb{S}\mathsf{G}_2\Big(\mathbb{S}\mathsf{G}_3\Big(\ldots\big(\mathbb{S}\mathsf{G}_n(\mathbf{X})\big)\Big)\Big), \quad Y_1 \;=\; \bigoplus_{i=1}^{n} X_i \;=\; X,$$

which shows that applying $\mathbb{S}\mathsf{G}_n, \mathbb{S}\mathsf{G}_{n-1}, \ldots, \mathbb{S}\mathsf{G}_2$ in sequence removes all masks and reveals the secret. The flaw arises because the gadget's output is effectively an $(n-1)$-sharing. □

5.2 Lemma 1 in Bricklayer Designs

In this subsection, we investigate how Lemma 1 applies in the practical setting of a bricklayer design, where a b-bit state $S = S^1 \parallel \cdots \parallel S^t$ is processed in t chunks, each of length m bits (so $b = t \times m$). A typical round-based operation on S proceeds as follows:

$$
\begin{aligned}
&r = 1 \\
&\mathsf{For}\ (r \leq \text{number of rounds}) \\
&\quad S \leftarrow \mathsf{S\text{-}box}(S^1) \parallel \cdots \parallel \mathsf{S\text{-}box}(S^t), \\
&\quad S \leftarrow \mathsf{Diffusion}(S), \\
&\quad r \leftarrow r + 1.
\end{aligned}
$$

In the protected (masked) version, the b-bit state S is replaced by an (nb)-bit n-shared state $\mathbf{S}$, where each bit of S is independently shared. The S-box layer is replaced by $\mathbb{S}$S-box gadgets, and the diffusion layer, being additive-affine, is straightforward to mask. Hence, the protected circuit follows:

$$
\begin{aligned}
&r = 1 \\
&\mathsf{For}\ (r \leq \text{number of rounds}) \\
&\quad \mathbf{S} \leftarrow \mathbb{S}\mathsf{S\text{-}box}(\mathbf{S}^1) \parallel \cdots \parallel \mathbb{S}\mathsf{S\text{-}box}(\mathbf{S}^t), \\
&\quad \mathbf{S} \leftarrow \mathbb{S}\mathsf{Diffusion}(\mathbf{S}), \\
&\quad r \leftarrow r + 1.
\end{aligned}
$$

To apply Lemma 1 in this context, we must verify that in each round of the bricklayer design, the inputs to the gadgets remain jointly independent n-sharings. Once this condition is satisfied, the entire circuit inherits first-order probing security by direct application of the lemma.

Lemma 3. *If* $\mathbb{S}$S-box *is first-order probing secure, and on m independent n-shared inputs produces m independent n-shared outputs—and if the diffusion layer is bijective—then the resulting bricklayer design is first-order probing secure.*

Proof. By construction, the initial shared state forms b independent n-sharings, which feed into t separate $\mathbb{S}$S-box gadgets. By the lemma's assumption, these $\mathbb{S}$S-box gadgets produce m independent n-sharings each, so overall they yield $b = t \times m$ independent n-sharings for the input to $\mathbb{S}$Diffusion. Because Diffusion is invertible, Lemma 2 ensures it preserves the independence of its input n-sharings. Hence, at each gadget in the design, the input n-sharings meet the same security conditions as in the standalone scenario, and by Lemma 1, the entire design is first-order probing secure. □

6 Higher-Order Probing Security in Round-Based Compositions

Achieving higher-order probing security (i.e., $d > 1$) for a round-based cipher with deterministic masking requires more than just the bijectivity of each round. To illustrate this, consider the serial composition of gadgets in Figure 4, where each gadget is second-order ($d = 2$) probing secure in isolation.

When instantiated with a second-order probing-secure $\mathbb{S}$S-box, we observe that placing one probe at the input of the first gadget and a second at the output of the rth gadget (for large r) reveals statistical dependence on the underlying secrets.[7] This demonstrates

[7] In our specific experiment, we used the SM2-$\mathbb{S}\chi_5$ gadget with $n = 3$ shares for the χ_5 component of Ascon. See Section 7.2.1 and Appendix B.2.

that simple serial composition—without an intervening diffusion layer—does *not* preserve second-order probing security.

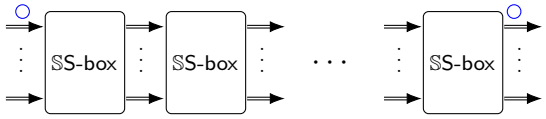


Figure 4: Serial composition of r gadgets. The blue circles indicate the probe locations in our numerical experiment.

Nevertheless, if each round transformation follows a bricklayer architecture, one can overcome this composition challenge by showing that the diffusion layer effectively behaves like a refresh gadget—up to a certain number of probes—and prevents an adversary from gaining any advantage by placing probes across different rounds. This observation calls for a closer examination of the diffusion layer's structure.

6.1 Properties of the Diffusion Layer

A diffusion layer is an invertible linear transformation that operates on a b-bit state S. This transformation can be represented using matrix multiplication as $S_{1\times b} \leftarrow S_{1\times b}\mathbf{M}_{b\times b}$. The primary design goal of the diffusion layer is to combine state bits through the XOR operation. The cryptographic effectiveness of the diffusion layer is typically evaluated using two metrics: the linear and differential *branch numbers*, denoted respectively by $\mathcal{B}_l$ and $\mathcal{B}_d$.

Definition 5 (Branch Numbers [DR20])**.** The branch numbers $\mathcal{B}_l$ and $\mathcal{B}_d$ are defined as:

$$\mathcal{B}_l = \min_{S\neq 0^b} \left[\mathsf{wt}(S) + \mathsf{wt}(S\mathbf{M}^\top)\right], \quad \mathcal{B}_d = \min_{S\neq 0^b} \left[\mathsf{wt}(S) + \mathsf{wt}(S\mathbf{M})\right], \tag{2}$$

where $\mathbf{M}^\top$ is the transpose of $\mathbf{M}$, and wt computes the number of non-zero (active) elements in its input vector.

The metric $\mathcal{B}_d$ identifies the minimum Hamming weight of input/output pairs and specifies the least number of active output bits resulting from a single active input bit. Conversely, $\mathcal{B}_l$ determines the minimum Hamming weight of parity relations between inputs and outputs. For this work, the $\mathcal{B}_l$ metric is of greater relevance.

Cryptographic View. The metrics $\mathcal{B}_l$ and $\mathcal{B}_d$ are directly related to the strength of a cipher against linear and differential cryptanalysis, respectively. Higher values for these branch numbers indicate increased resistance to such attacks. However, achieving higher branch numbers typically comes at the cost of increased computational complexity.

Definition 6 (Complexity of Diffusion Layer [JPST17])**.** The average number of XOR operations required to compute each output bit is defined as the complexity of the diffusion layer.

For Ascon, the linear and differential branch numbers of the diffusion layer are both 4 [DEMS21]. Specifically, each input bit affects three output bits, and no parity relation with fewer than four bits exists. This property can be verified by inspecting Equation (4). It is noteworthy that, for Ascon, the branch number is relatively low.[8] With the relatively low branch number, the diffusion layer's computational complexity is also very low: each output bit is computed using just 2 XOR operations.

[8] Since a single active input can create at most b active outputs, branch numbers are upper-bounded by $b+1$. In practice, this upper bound (or values close to it) is achievable for a special class of $\mathbf{M}$ matrices described by maximum distance separable (MDS) codes [DR20].

Polynomial View. By representing the b binary random variables of the input state as $\{X_1, \ldots, X_b\}$, we can express the output bits of the diffusion layer as polynomials:

$$f_j \;=\; \bigoplus_{i=1}^{b} m_i^j \, X_i,$$

for $1 \leq j \leq b$. Here, the (binary) coefficients m_i^j are elements of the matrix $\mathbf{M}$, with $m_i^j = \mathbf{M}[i, j]$. Identifying $\mathcal{B}_l$ then reduces to finding a parity relation in the set

$$\mathcal{F} \;=\; \{X_1, \ldots, X_b,\; f_1, \ldots, f_b\}$$

with the smallest Hamming weight. A *parity relation* is a subset of $\mathcal{F}$ whose elements (bits) sum to zero (see Section 2). Formally, we have:

Lemma 4. *With f_j polynomials defined as above, and their collection given in $\mathcal{F}$, one can compute $\mathcal{B}_l$ as*

$$\mathcal{B}_l \;=\; \min_{|\mathcal{C}|} \Big[\bigoplus_{l \in \mathcal{C}} \mathcal{F}[l] \;=\; 0 \Big], \tag{3}$$

where $\mathcal{F}[l]$ denotes the l-th element of $\mathcal{F}$, and $\mathcal{C}$ is a non-empty set of indices.

Proof. The proof can be found in Appendix A. □

6.1.1 Self-Composition of the Diffusion Layer

The evaluation of the diffusion mapping is just one part of the computations in a single cipher round. A cipher typically operates over multiple rounds, with non-linear operations interleaved between successive diffusion layers. To gain insight into how parity relations evolve with an increasing number of rounds, we temporarily disregard the presence of non-linear layers and study the composition of the diffusion transformation itself. The composition is computed as $(S\mathbf{M})\mathbf{M} = S\mathbf{M}^2$. In general, the composition over r rounds is directly represented by the matrix $\mathbf{M}^r$.

Definition 7 (Order of the Diffusion Layer)**.** The *order* of the diffusion layer is defined as the number of rounds r for which the layer composes with itself to produce the identity mapping. Formally, it is the smallest integer r such that $\mathbf{M}^r = \mathbf{I}$.

Composing the diffusion layer introduces new parity relations between the input S and the outputs $S\mathbf{M}^r$ (for varying r). Even before reaching the order, these new relations may have weights smaller than $\mathcal{B}_l$. More specifically, these relations arise in the $(r+1)b$-element set of polynomials:

$$\mathcal{F}^r = \{X_1, \ldots, X_b, f_1, \ldots, f_b, f_{b+1}, \ldots, f_{2b}, \ldots, f_{(r-1)b+1}, \ldots, f_{rb}\},$$

where, for $0 \leq \alpha < r$ and $1 \leq \beta \leq b$, $f_{\alpha b+\beta}$ is defined as:

$$f_{\alpha b+\beta} = \bigoplus_{i=1}^{b} \mathbf{M}^{\alpha+1}[\beta, i] X_i.$$

We denote the minimum weight of parity relations in the set $\mathcal{F}^r$ as $\mathcal{B}_l^r$. Note that $\mathcal{B}_l^r$ is a decreasing function of the number of rounds r, and at the order of the layer, it reaches the minimum value $\mathcal{B}_l^r = 2$.

Example 4. Let $b = 15$ and the diffusion be defined by the following relation:

$$\mathbf{X} \leftarrow \mathbf{X} \oplus (\mathbf{X} \lll 1) \oplus (\mathbf{X} \lll 3),$$

where $\mathbf{X}$ is a vector of 15 bits. Using a computer program, we verified that this map is invertible, has an order of 15, and its $\mathcal{B}_l$ is 4.[9]

For this map, we constructed the set $\mathcal{F}^r$ and computed the corresponding $\mathcal{B}_l^r$ by exhaustively testing all subsets of $\mathcal{F}^r$ with fewer than $\mathcal{B}_l$ entries. We observed that, up to the order, the minimum weight of parity relations does not fall below the initial branch number. □

We note that for diffusion layers used in ciphers, the order is expected to be practically high. Moreover, $\mathcal{B}_l^r = \mathcal{B}_l$ is typically maintained for sufficiently high values of r in practical applications.

6.2 Joint Independence Across the Diffusion Layer

Consider a toy 3-bit linear transformation with the following input-output relation:

$$(X, Y, Z) \to (X \oplus Y, Y \oplus Z, X \oplus Y \oplus Z).$$

For this map, we have $\mathcal{B}_l = 3$. In this example, we can directly verify that the knowledge of a single input bit does not determine any of the output bits. More precisely, any two variables chosen from the inputs and outputs are jointly independent. The following lemma generalizes this observation.

Lemma 5. *Let $\mathcal{B}_l$ denote the branch number of a diffusion layer. Then, $\mathcal{B}_l - 1$ probes placed on the inputs and outputs of* Diffusion *are jointly independent. Alternatively, the collection of all probes on the inputs and outputs is $(\mathcal{B}_l - 1)$-jointly independent.*

Proof. Assume these probes (variables) are dependent. Then, there must exist a parity relation between the inputs and outputs with fewer than $\mathcal{B}_l$ terms. This contradicts the definition of $\mathcal{B}_l$. □

Lemma 5 can be extended to the composition of diffusion layers: any $\mathcal{B}_l^r - 1$ probes placed on the inputs and outputs of the diffusion layer in r rounds are jointly independent.

Joint Independence of n-Sharings Across the Diffusion Layer. We have thus far shown that the diffusion layer provides a measure of joint independence among the *output* natives, assuming the *inputs* are unknown, uniform, and independent. However, this result, in its current form, is not necessarily useful in practice, because we cannot guarantee that every input meets these assumptions (e.g., some natives like IVs or nonces, or their initial-round combinations, may be fixed or public). By shifting to a masked setting and considering n-shared inputs instead of natives, we arrive at the following more practical statement:

Lemma 6. *Any collection of $(\mathcal{B}_l - 1)$ n-sharings at the input and output of* $\mathbb{S}$Diffusion *is jointly independent. Equivalently, all input and output n-sharings together form a set of $(\mathcal{B}_l - 1)$-jointly independent n-sharings.*

Proof. Let $\mathcal{W} = \{\mathbf{X}^1, \ldots, \mathbf{X}^{\mathcal{B}_l - 1}\}$ be a collection of n-sharings. Because each share index is processed by an identical, separate circuit, any dependency in $\mathcal{W}$ would imply that, for each share index i, the set $\mathcal{W}_i = \{X_i^1, \ldots, X_i^{\mathcal{B}_l - 1}\}$ must also be dependent. Therefore, for some subset $\mathcal{V} \subseteq \{1, \ldots, \mathcal{B}_l - 1\}$, we would have

$$\bigoplus_{j \in \mathcal{V}} X_i^j \;=\; 0,$$

[9] This map can be expressed with *circulant matrices*, for which there is a straightforward approach to compute the order and inverse [LS16].

which, when summed over all share indices $1 \leq i \leq n$, implies

$$\bigoplus_{j \in \mathcal{V}} X^j \;=\; 0,$$

where this relation holds on the *native* random variables. However, by definition of $\mathcal{B}_l$, no parity relation among the native variables can have fewer than $\mathcal{B}_l$ terms. Hence, such a dependency in $\mathcal{W}$ cannot occur.

Finally, note that n-sharing independence (Definition 3) places no constraints on the values of the underlying natives. In particular, this lemma still holds if all input natives are zero. □

Joint Independence of n-Sharings at the $\mathbb{S}$S-box Level. The input and output wires of $\mathbb{S}$Diffusion are grouped according to the adjacent $\mathbb{S}$S-box layers. For our analysis, it is important to determine how many of these groups—each corresponding to one $\mathbb{S}$S-box—remain jointly independent. We refer to this metric as $(\mathcal{B}_{l\text{-box}} - 1)$, where $\mathcal{B}_{l\text{-box}}$ is the *box linear branch number*.

Previously, we showed that up to $(\mathcal{B}_l - 1)$ individual n-sharings across the diffusion layer are jointly independent. Given that each S-box operates on m bits, the number of jointly independent $\mathbb{S}$S-box inputs/outputs is bounded as follows:

$$\frac{\mathcal{B}_l - 1}{m} \;\leq\; (\mathcal{B}_{l\text{-box}} - 1) \;\leq\; (\mathcal{B}_l - 1).$$

The exact value of $\mathcal{B}_{l\text{-box}}$ can be computed as follows:

Definition 8 (Box Linear Branch Number)**.** Let S be a b-bit input to the diffusion layer, and let $\mathbf{M}^\top$ denote the transpose of the matrix defining this layer (as described in Section 6.1). The *box linear branch number* $\mathcal{B}_{l\text{-box}}$ is defined as

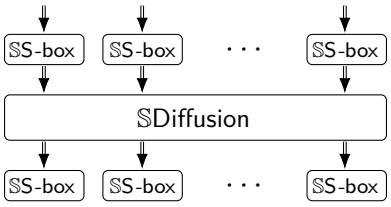
$$\mathcal{B}_{l\text{-box}} = \min_{S \neq 0^b} \left[\#\text{active S-boxes in } S \;+\; \#\text{active S-boxes in } S\mathbf{M}^\top \right],$$

where an S-box is considered active if any of its m bits is active.

6.3 Our Higher-Order Composition Rule

While our motivating case involves deterministic masking, the result itself holds more broadly. We now present our main composition result, which underlies deterministic masking schemes but applies more generally to any probing-secure gadgets.

Theorem 1 (Composition Theorem)**.** *Consider two consecutive* $\mathbb{S}$S-box *layers separated by a single* $\mathbb{S}$Diffusion *layer:*



Assume the following conditions hold:

- *Each* $\mathbb{S}$S-box *is* $d_{\mathbb{S}\text{S-box}}$*-probing secure.*
- *The* $\mathbb{S}$Diffusion *layer is bijective.*

- *Given independent n-sharings as input, both* $\mathbb{S}$SS-box *and* $\mathbb{S}$Diffusion *produce independent n-sharings at their output.*

- *Let* $\mathcal{B}_{l\text{-box}}$ *denote the box linear branch number of the diffusion layer.*

Then, the probing security order d_{comb} *of the combined structure satisfies:*

$$\min\{d_{\mathbb{S}\text{S-box}},\ (\mathcal{B}_{l\text{-box}}-1)\}\ \leq\ d_{comb}\ \leq\ d_{\mathbb{S}\text{S-box}}.$$

Proof. We prove the theorem in four steps.

Step 1: Probes on $\mathbb{S}$S-box Gadgets Only. Assume all probes are placed on the $\mathbb{S}$S-box gadgets, and none target wires inside the $\mathbb{S}$Diffusion layer. Let the probes be distributed across a set $\mathcal{T}$ of $\mathbb{S}$S-box gadgets. If $|\mathcal{T}| \leq (\mathcal{B}_{l\text{-box}}-1)$, then the inputs and outputs of these gadgets remain jointly independent. Therefore, each gadget behaves as in the standalone case, preserving its local $d_{\mathbb{S}\text{S-box}}$-order security.

Step 2: Total Number of Probes. We now determine how many probes these $|\mathcal{T}|$ gadgets can collectively tolerate. There are two cases:

- If $(\mathcal{B}_{l\text{-box}}-1) \geq d_{\mathbb{S}\text{S-box}}$, then the limiting factor is the local security of the gadgets. Thus, $d_{\text{comb}} = d_{\mathbb{S}\text{S-box}}$.

- If $(\mathcal{B}_{l\text{-box}}-1) < d_{\mathbb{S}\text{S-box}}$, then even though each gadget can tolerate more probes, joint independence across too many gadgets may not hold. Therefore, we can guarantee:

$$(\mathcal{B}_{l\text{-box}}-1) \leq d_{\text{comb}} \leq d_{\mathbb{S}\text{S-box}}.$$

Hence, in all cases,

$$\min\{\, d_{\mathbb{S}\text{S-box}},\ (\mathcal{B}_{l\text{-box}}-1)\}\ \leq\ d_{\text{comb}}\ \leq\ d_{\mathbb{S}\text{S-box}}.$$

Step 3: Probes on $\mathbb{S}$Diffusion. Now consider probes placed on the internal wires of $\mathbb{S}$Diffusion. We use the probe elimination strategy (see Section 4.3): any such probe can be eliminated using the independent offline randomness from the output of one $\mathbb{S}$S-box.

For example, if a wire carries $P = X_i \oplus P'$, where X_i is a share from the output of $\mathbb{S}\text{S-box}_j$ (see Figure 5), then P can be eliminated if:

1. X_i does not appear in P'.

2. X_i is uniformly distributed.

3. X_i is independent of all other probes.

4. Not all shares of the native variable corresponding to X_i are involved in P'.

Thanks to the linearity of $\mathbb{S}$Diffusion (no cross-share operations), Conditions (1) and (4) are automatically satisfied. Condition (2) follows because output of gadgets are n-sharing, which means that any $n-1$ collection of shares are uniformly distributed. To ensure Condition (3), we add $\mathbb{S}\text{S-box}_j$ to $\mathcal{T}$. If $|\mathcal{T}| \leq (\mathcal{B}_{l\text{-box}}-1)$, then this inclusion does not break the independence assumptions.

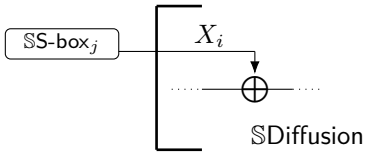


Figure 5: A probe on $\mathbb{S}$Diffusion is reduced to a probe on $\mathbb{S}$S-box$_j$.

Step 4: Final Bound. Each $\mathbb{S}$S-box added to $\mathcal{T}$ corresponds to at most one probe (direct or via diffusion). As long as $|\mathcal{T}| \leq (\mathcal{B}_{l\text{-box}} - 1)$, we can ensure that the gadgets operate on jointly independent inputs. From Step 2, this yields:

$$\min\{\, d_{\mathbb{S}\text{S-box}},\, (\mathcal{B}_{l\text{-box}} - 1)\} \;\leq\; d_{\text{comb}} \;\leq\; d_{\mathbb{S}\text{S-box}}.$$

This completes the proof. □

In what follows, we will use $d_{\mathbb{S}\text{S-box}}$ and d_{comb} to denote probing security orders where ambiguity might arise; otherwise, we simply use d.

Security Claim for Multiple Rounds. In practical cipher designs, the combination of two consecutive rounds—connected via a single diffusion layer—typically determines the limiting value of $\mathcal{B}_{l\text{-box}}$. Adding additional rounds generally does not further reduce this quantity. This behavior reflects the *avalanche effect*, a common design goal in cryptographic primitives, where changes to a small number of input bits rapidly propagate across the state.

As a result, the compositional guarantees established in Theorem 1 often extend naturally to larger numbers of rounds. However, we leave a formal treatment of this extension to future work.

Use of Online Randomness in $\mathbb{S}$S-box. The proof of Theorem 1 assumes that each $\mathbb{S}$S-box produces independent n-sharings at its output given independent n-shared inputs, and that it satisfies local probing security of order $d_{\text{S-box}}$. However, the theorem imposes no constraints on how these properties are achieved—particularly, it does not require that the computation inside the gadget be deterministic. This flexibility allows the composition result to hold even when the $\mathbb{S}$S-box uses *online* randomness in its internal operations. This generality significantly enhances the practical applicability of our composability theorem.

Application to AES. In the case of AES, the combination of the ShiftRows and MixColumns layers yields a box linear branch number of $\mathcal{B}_{l\text{-box}} = 5$. Therefore, the diffusion layer in AES can support up to $d = \mathcal{B}_{l\text{-box}} - 1 = 4$ probing security order—*provided* that the $\mathbb{S}$S-box meets the requirements of Theorem 1.

6.3.1 Towards Probing-Secure Randomness Reuse

So far, our discussion has assumed that secure computation inside each $\mathbb{S}$S-box is either deterministic—as demonstrated in this work for the case of Ascon—or that each gadget receives its own fresh (online) randomness. A complementary paradigm aims to minimize the need for online randomness by enabling its *reuse* across rounds.

Figure 6 illustrates a conceptual architecture where auxiliary randomness, introduced only in the initial round, is propagated forward via a parallel *randomness network*. This structure supports the reuse of offline randomness while aiming to maintain a high level of probing security.

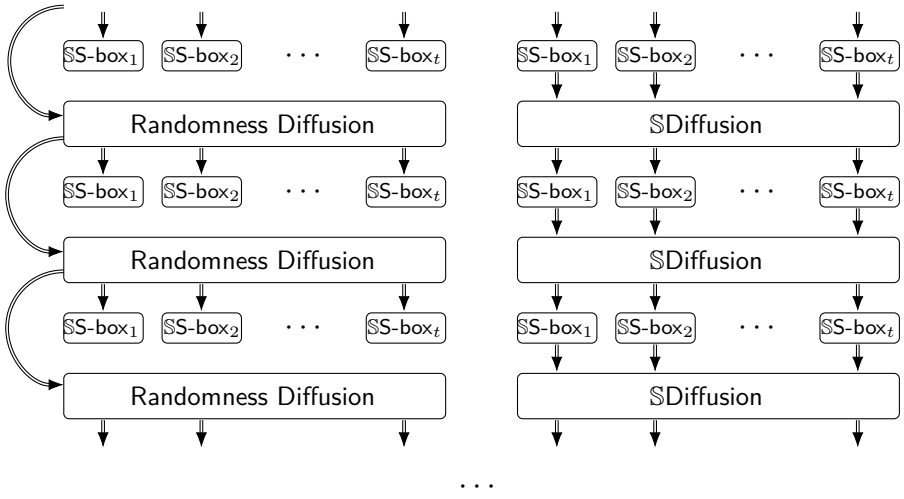


Figure 6: A parallel randomness reuse network that mirrors the round structure of the masked cipher. The initial invocation of Randomness Diffusion receives fresh randomness; subsequent rounds reuse the transformed outputs.

The problem of expanding a small amount of true randomness into a larger amount of pseudo-random bits suitable for side-channel protection has been previously considered in the literature [CGZ20, IKL+13]. However, the specific architecture depicted in Figure 6—which mirrors the round structure of the cipher while avoiding non-linear operations—has not been studied. Such a structure may offer both implementation simplicity and improved efficiency.

Mathematically, the behavior of the randomness reuse network mirrors that of diffusion layer self-composition, as discussed in Section 6.1.1. There, we examined how to determine the number of jointly independent outputs at the bit level. Here, a similar analysis can be extended to groupings that align with S-box boundaries.

Modeling the Problem. Let $\mathbf{M}_R$ and $\mathbf{M}_C$ represent the diffusion matrices for the randomness network and the cipher, respectively. The inputs to each $\mathbb{S}$S-box derive from the outputs of both $\mathbf{M}_R$ and $\mathbf{M}_C$. To evaluate how many $\mathbb{S}$S-boxes receive jointly independent inputs, we define the following set of polynomials over the inputs $\{X_1, \ldots, X_b\}$:

$$\mathcal{F}^r = \{X_1, \ldots, X_b,\ g_1, \ldots, g_b,\ f_1, \ldots, f_{rb}\},$$

where the polynomials are defined as follows:

$$g_\beta = \bigoplus_{i=1}^{b} \mathbf{M}_C[\beta, i]X_i, \quad f_{\alpha b+\beta} = \bigoplus_{i=1}^{b} \mathbf{M}_R^{\alpha+1}[\beta, i]X_i,$$

for $1 \leq \beta \leq b$ and $0 \leq \alpha < r$. We assume that the input lengths match; otherwise, missing inputs in $\mathbf{M}_R$ can be accommodated by inserting zero columns at the appropriate positions to align with the indexing of the X_i. In this setup, $f_{\alpha b+\beta}$ polynomials are computing the randomness that are delivered to the $\mathbb{S}$S-boxes at round $\alpha + 1$, and g_β is representing possible mix of the randomness via the masked cipher round.

We define $\mathcal{B}^r_{l\text{-box}}$ as the minimal number of S-boxes involved in a parity relation over $\mathcal{F}^r$. Let each $\mathbb{S}$S-box be $d_{\mathbb{S}\text{S-box}}$-secure and produce n-shared outputs from n-shared inputs for any fixed auxiliary randomness.

We conjecture that the probing security order d_{comb} of the r-round composition satisfies:

$$\min\{\, d_{\mathbb{S}\text{-box}},\, (\mathcal{B}^r_{l\text{-box}} - 1)\} \;\leq\; d_{\text{comb}} \;\leq\; d_{\mathbb{S}\text{-box}}.$$

This conjecture is based on the structural analogy with our deterministic composition result (Theorem 1), where $\mathcal{B}_{l\text{-box}}$ governs the effective independence across masked components. Extending the same reasoning to the auxiliary randomness setting appears promising, but proving this bound remains an open problem. We leave its rigorous analysis to future work.

7 Case Study on Ascon

In this section, we briefly review the structure of Ascon and investigate the feasibility of applying deterministic masking to it. The value of its box linear branch number is $\mathcal{B}_{l\text{-box}} = 4$, which permits probing security up to order $d = 3$.

In this work, we focus on designing and analyzing $\mathbb{S}$S-box gadgets for security orders $d \in \{1, 2\}$. A construction for $d = 3$ based on the composability framework presented here has been introduced and analyzed in [JSB25].

For the specific case of $d = 2$, we additionally present an alternative, *direct* method for verifying the probing security of the round composition.

We conclude this section by discussing the resulting randomness savings and reporting on software implementations of the protected Ascon design for $d \in \{1, 2\}$.

7.1 Ascon's Structure

Ascon [NIS24, DEMS21] has a bricklayer structure operating on a state of $b = 320$ bits. It employs a *sponge-based* [BDPV07] mode of operation for authenticated encryption. The key, tag, and nonce are each 128 bits. As outlined in Figure 7, the encryption process is divided into four phases: (1) *Initialization*: The state is initialized with the (private) key K, the (public) nonce N, and a fixed initialization vector (IV). (2) *Associated Data Processing*: The state is updated with associated data blocks A_i. Associated data is not confidential and is therefore not encrypted. However, it is absorbed by the sponge to verify its integrity during transit. (3) *Plaintext Processing*: Plaintext blocks P_i are injected into the state, and ciphertext blocks C_i are extracted. (4) *Finalization*: The key K is injected again, and the tag T is extracted for authentication.

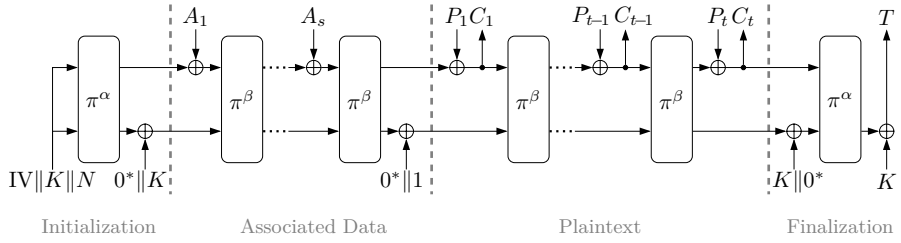


Figure 7: Ascon. π^α and π^β are application of the permutation π in α and β rounds.

Ascon's Permutation. The cryptographic permutation of Ascon is called Ascon-p, which we here for continence denote with π. The permutation π is an iterative application of a round transformation on the 320-bit state $\mathbf{s}$. The state is represented as a 5×64 table, $\mathbf{s} = [\mathbf{s}_1, \mathbf{s}_2, \mathbf{s}_3, \mathbf{s}_4, \mathbf{s}_5]^\top$, where each $\mathbf{s}_i$ is 64 bits long. The round transformation

comprises three steps: (1) *Addition of Round Constants*: XORs a round-specific 1-byte constant to the state. (2) *Non-linear Layer*: Applies a 5-bit S-box 64 times in parallel. For $1 \leq i \leq 64$, the S-box is applied to $(\mathbf{s}_1[i], \mathbf{s}_2[i], \mathbf{s}_3[i], \mathbf{s}_4[i], \mathbf{s}_5[i])$, where $[i]$ denotes the i-th bit. (3) *Diffusion Layer*: XOR rotated parts of the state. Specifically, for $1 \leq j \leq 5$, $\mathbf{s}_j$ is updated as:

$$\mathbf{s}_j \leftarrow \mathbf{s}_j \oplus (\mathbf{s}_j \lll \theta_j) \oplus (\mathbf{s}_j \lll \theta'_j), \tag{4}$$

where the rotation amounts θ_j and θ'_j are specified in [DEMS21].

Ascon's S-box. The S-box is χ_5 (defined below) between two thin 5-bit linear and affine layers, as depicted in Figure 8. The round function contains 64 S-boxes, one for each i.

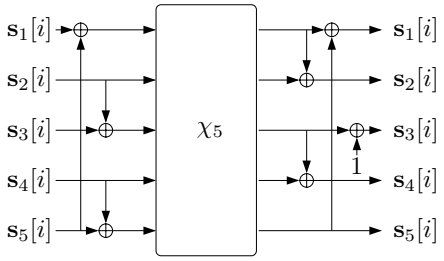


Figure 8: Ascon's S-box.

The Map χ_m. The non-linear map $\chi_m \colon \{0,1\}^m \to \{0,1\}^m$ is defined as follows: For binary input vector $(X^1, \ldots, X^m)$, the output Y^i, for $1 \leq i \leq m$, is computed as:

$$Y^i = X^i \oplus (1 \oplus X^{i+1})X^{i+2},$$

where the boundary values X^{m+1} and X^{m+2} are substituted by X^1 and X^2, respectively.

The map χ_m was investigated by Daemen in [Dae95]. For odd m, it is bijective. This map is deployed in several other important cryptographic algorithms. For instance, χ_5 is used in SHA-3 [SHA15] and χ_3 is used in Xoodyak [DHP+20]. Notably, in these algorithms, χ_m provides the only source of non-linearity in the round function.

Protecting the Permutation. The addition of the key K at the end of the *initialization* phase and the start of the *finalization* phase, as shown in Figure 7, prevents side-channel state recovery attacks (carried out during Associated Data and Plaintext Processing) from escalating into *key recovery* or *tag forgery* attacks [DEMS21, BBC+20].

To specifically mitigate key recovery attacks, only the initialization and finalization phases require protection. Considering that both phases consist of a single invocation of the permutation π, our focus will be on securing the computations of π.

7.2 Protected Implementation of the Permutation

The permutation consists of two main building blocks: the Diffusion layer and the S-box. While Diffusion is affine and thus straightforward to mask at any order, the non-linear S-box presents a greater challenge. In the following, we focus on existing proposals for *deterministic* masking of the S-box for orders $d \in \{1, 2\}$.

7.2.1 Ascon's Deterministic $\mathbb{S}$S-boxes

Ascon's S-box (Figure 8) implements the χ_5 function, enclosed between two linear-affine layers. In the corresponding $\mathbb{S}$S-box construction, these layers are trivially masked using standard Boolean masking techniques (Section 3.2) and then composed with a $\mathbb{S}\chi_5$ gadget.

For $n \in \{2, 3\}$, the literature provides deterministic, $(n-1)$-order probing-secure, and bijective $\mathbb{S}\chi_5$ gadgets that preserve the independence of n-shared outputs when fed with independent n-shared inputs. We briefly review these gadgets below.

Dae-$\mathbb{S}\chi_5$ Gadget. Daemen et al. [DDE+20] introduced a two-share, first-order secure $\mathbb{S}\chi_5$ gadget, denoted Dae-$\mathbb{S}\chi_5$. They formally verified the security of the resulting $\mathbb{S}$S-box (built using this gadget) using the maskVerif tool [BBD+15a, BBC+19].[10]

This design has been implemented by the official Ascon team in a software setting,[11] using a *bit-sliced* representation combined with *share rotation* to enhance resistance against side-channel attacks [GMSP23].

Although Dae-$\mathbb{S}\chi_5$ requires one bit of randomness, it is still considered deterministic because no *online* randomness is used during the computation; the randomness is injected only once at initialization and reused in later rounds. In addition to its first-order probing security, Dae-$\mathbb{S}\chi_5$ also achieves *transition security*. The inclusion of an auxiliary random variable guarantees bijectivity, mapping all 2^{11} possible inputs (i.e., $2 \times 5 + 1 = 11$ bits) to unique outputs.

SM-$\mathbb{S}\chi_5$ Gadgets. Shahmirzadi and Moradi [SM21] employed a computerized search to identify $\mathbb{S}\chi_5$ gadgets for $(n = 2, d = 1)$ and $(n = 3, d = 2)$. Their constructions provide glitch resistance and achieve probing security without requiring any auxiliary randomness. We refer to these designs as SM1-$\mathbb{S}\chi_5$ and SM2-$\mathbb{S}\chi_5$, respectively. Slightly simplified versions of their internal computations, adapted for a probing-security focus, are provided in Appendices B.1 and B.2.

Using the SILVER tool [KSM20], we verified the probing security and bijectivity of these SM-$\mathbb{S}\chi_5$ gadgets, as well as the full $\mathbb{S}$S-box constructions that incorporate them. These gadgets are also used in our practical side-channel leakage evaluations.

7.3 Application of Our Composition Rules

The $\mathbb{S}$S-box constructions based on SM-$\mathbb{S}\chi_5$ satisfy the conditions of Lemma 3, and thus provide first-order probing security by composition. In particular, the gadget SM2-$\mathbb{S}\chi_5$ achieves $d_{\mathbb{S}\text{S-box}} = 2$. Hence, when used within a bricklayer design, the achievable probing security order d_{comb} satisfies:

$$\min\{2, 3\} \;\leq\; d_{\text{comb}} \;\leq\; 2,$$

which implies $d_{\text{comb}} = 2$.

We additionally present an alternative, direct argument based on probe elimination to support the probing security of Ascon's permutation across multiple rounds. The key observation is that when using SM2-$\mathbb{S}\chi_5$ gadgets, it is possible to derive a representation that decouples the transformation of shares in each round from the evolution of their underlying native variables (secrets). Specifically, while the unshared native state evolves non-linearly, the corresponding shares undergo only linear transformations. This separation simplifies the task of identifying how each $\mathbb{S}$S-box input remains blinded by the randomness in the initial input shares throughout all rounds.

[10] At the time of verification, maskVerif followed the methodology of [BBD+15a]. The more recent version [BBC+19], which relies on simulation-based rules, cannot verify the probing security of Dae-$\mathbb{S}\chi_5$ or its associated $\mathbb{S}$S-box.

[11] https://github.com/ascon/simpleserial-ascon

7.4 Direct Analysis of the $d = 2$ Case

Linear Representation of Output Shares. A key property of the SM2-$\mathbb{S}\chi_5$ gadget (Appendix B.2) is that, for a *fixed* native state, each output share can be written as a linear function of the input shares. For example, the output share A'_1 satisfies:

$$A'_1 = C_1B_1 \oplus C_2 \oplus C_2B_1 \oplus C_3 \oplus A_1 \oplus C_3B_1,$$

which simplifies to

$$A'_1 = CB_1 \oplus C_2 \oplus C_3 \oplus A_1,$$

where $C = C_1 \oplus C_2 \oplus C_3$ is a fixed function of the native state. Thus, the expression is linear in the input shares when the state is treated as constant.

Adopting the convention that native variables (e.g., C) are functions of the initial state S, we can rewrite A'_1 schematically as:

$$A'_1 = \big(A_1 \oplus C_2 \oplus C_3\big) \ \oplus \ \big(B_1 \cdot f(S)\big),$$

where $f(S)$ denotes some function of the native state whose exact form is not tracked.

This structure holds for all output shares of the SM2-$\mathbb{S}\chi_5$ gadget. In what follows, we exploit this linearity to trace how the shares evolve across rounds. From a side-channel perspective, the precise non-linear evolution of the native state S is unimportant—as long as the shares correctly mask the state, the security objective is met.

Tracing Round-Input Formats. Ascon's permutation operates on a 320-bit state ($b = 320$). Under an $n = 3$ sharing, the protected state comprises $nb = 960$ shares. For notational convenience, we label these shares as $\{X_1, X_2, \ldots, X_{nb}\}$, grouping together the shares of each bit in the native state.

To track how these shares propagate through the layers of $\mathbb{S}$S-box and $\mathbb{S}$Diffusion, we model each input share Y to round r as a polynomial expression of the form:

$$Y \ = \ \left(\bigoplus_{i \in \mathcal{I}} X_i\right) \ \oplus \ \left(\bigoplus_{j \in \mathcal{J}} X_j \cdot f_j(S)\right), \tag{5}$$

where $\mathcal{I}, \mathcal{J} \subseteq \{1, \ldots, nb\}$ are disjoint sets of indices, and each $f_j(S)$ is a Boolean function of the native state S. Since these functions serve only to modulate the input shares via native-state dependence, we do not track their explicit forms. Instead, our analysis follows only the indices $\mathcal{I}$ and $\mathcal{J}$.

Blinding with Initial Input Shares. Due to the symmetry of the cipher structure, we focus our second-order security analysis on showing that the n-shared inputs to any $\mathbb{S}$S-box gadget beyond the first are independent of the initial inputs to the first one. Specifically, we consider the initial 15 shares $\{X_1, \ldots, X_{15}\}$ that serve as input to the first $\mathbb{S}$S-box, and we aim to show that later-round inputs are statistically independent of these shares and of the state S.

We do this by inspecting the expressions in Equation (5). If, for a given Y, there exists at least one index $i \in \mathcal{I}$ such that $i \notin \{1, \ldots, 15\}$, then Y is blinded by a share independent of the first $\mathbb{S}$S-box input. When analyzing multiple shares Y, we further ensure that each is blinded by a *distinct* X_i in order to satisfy joint independence. This argument follows the probe-elimination reasoning from Section 4.

Note that only $n-1$ shares per native variable are uniformly random; using all n shares would compromise blinding. Hence, our approach ensures that each X_i used for blinding comes from a separate native variable or avoids exhausting all its shares.

We carried out this procedure for up to $r = 5$ rounds and verified that the inputs to all $\mathbb{S}$S-box gadgets remained independent of $\{X_1, \ldots, X_{15}\}$ and of the state S. For rounds beyond $r = 5$, each polynomial in Equation (5) effectively reduces to the following form:

$$Y = \bigoplus_{j=1}^{nb} X_j \cdot f_j(S), \tag{6}$$

where again the goal is to ensure that for some $j \notin \{1, \ldots, 15\}$, the function $f_j(S)$ evaluates to 1. Since each $f_j(S)$ is Boolean and state-dependent, the probability that all $f_j(S)$ vanish over $j \in \{16, \ldots, nb\}$ is negligibly small. Hence, with overwhelming probability, the share Y is blinded by a variable independent of $\{X_1, \ldots, X_{15}\}$, ensuring second-order security across multiple rounds.

7.5 Randomness Savings

In Ascon's masked implementation, the only non-linear component per round is the χ_5 layer. Each round processes the 320-bit state using 64 instances of χ_5, and each instance involves five AND operations. Thus, each round contains a total of 320 AND operations.

A typical $\mathbb{S}$AND gadget (e.g., from [ISW03, CS20]) requires $\frac{d(d+1)}{2}$ bits of *online* randomness per invocation to achieve d-order probing security. For Ascon, this leads to an online randomness cost of approximately $160\, d(d+1)$ bits *per round*, assuming randomness is only used within non-linear components. This cost scales linearly with the number of rounds.

In contrast, our deterministic masking approach eliminates this overhead entirely: no online randomness is required during the execution of the masked permutation. The initial input shares encapsulate all necessary randomness, and this offline entropy is recycled across rounds.

7.6 Implementation Results

We implemented the proposed deterministic masking scheme for Ascon.[12] Our starting point was the existing software-protected implementation of Ascon, available at [DDM+22], which we adapted to suit the deterministic masking framework presented in this paper.

Specifically, we replaced the original $\mathbb{S}\chi_5$ gadgets with the SM1-$\mathbb{S}\chi_5$ and SM2-$\mathbb{S}\chi_5$ constructions, targeting first- and second-order probing security, respectively. We eliminated all dependencies on auxiliary randomness, ensuring that the entire implementation relies solely on offline randomness embedded in the initial input shares. To prevent compiler optimizations from merging or simplifying masked values (e.g., by combining shares), we implemented the critical $\mathbb{S}\chi_5$ computations directly in ARM assembly. Unlike the original code base, our version is explicitly backed by a formal composability theorem.

Our implementation preserves several useful features from the original repository, including share rotation, device-specific leakage mitigations, and a side-channel secure tag comparison mechanism. Additionally, we introduced word-level interleaving within the $\mathbb{S}$S-box computations to reduce Hamming distance leakage stemming from register-level value transitions.

Execution and Measurement Setup. All experiments were performed on a ChipWhisperer CW308 board with an STM32F303 UFO target,[13] featuring a 32-bit ARM Cortex-M4 microcontroller. Power traces were captured using a ChipWhisperer-Lite at a sampling rate four times the device's clock frequency. Initial randomness for input sharing was generated

[12] https://github.com/vahid-jahandideh/deterministic-masking-ascon.git
[13] https://github.com/newaetech/chipwhisperer

using a modified `randombytes.c` file, based on the standard `rand()` and `srand()` functions from `stdlib.h`, as used in the original framework.

Share Rotation and Hamming Distance Leakage. Share rotation is a relatively recent and effective technique introduced in [GD23b] to mitigate Hamming distance leakage. The core idea is to ensure that different shares are loaded into registers with varying amounts of rotation. This reduces the likelihood that transitions between share values result in data-dependent leakage—especially in designs where no cross-share operations occur.

However, within non-linear computations such as the $\mathbb{S}$S-box, registers may be overwritten without proper share rotation, potentially introducing side-channel leakage. One common mitigation strategy is to explicitly clear registers (e.g., using `mov R0, 0`) before writing new values. Yet, clearing low-index registers (like `R0`–`R4`) can interfere with correct computation due to register allocation constraints.

To resolve this, our implementation combines share rotation with 32-bit *word-level interleaving* inside the 64-bit $\mathbb{S}$S-box computations. This approach eliminates visible leakage peaks in the T-test measurements, albeit at the cost of slightly increased cycle count.

7.6.1 Speed Comparison

Table 1 compares the running time (in terms of clock cycles per encrypted byte) for three protection levels: unprotected ($d = 0$), first-order ($d = 1$), and second-order ($d = 2$). We benchmark both our deterministic masking implementation and the reference implementation from [DDM+22]. While our approach eliminates the need for a randomness reuse network, the $\mathbb{S}$S-box computations are heavier and incorporate word-level interleaving. As a result, despite some architectural simplifications, our implementation consumes roughly 25% more cycles. Importantly, our design is backed by formal composability guarantees (Theorem 1) as well as direct analysis in Section 7.4, ensuring provable probing security for the composed permutation. The 25% overhead observed even in the unprotected ($d = 0$) case is primarily due to the word-level interleaving countermeasure.

Table 1: Cycle count per byte under different protection orders

Protection Order	Source	1 Byte	Long Message
$d = 0$	Ascon Team [DDM+22]	4,274	99
	This Work	5,109	124
$d = 1$	Ascon Team [DDM+22]	11,085	310
	This Work	13,339	388
$d = 2$	Ascon Team [DDM+22]	20,883	585
	This Work	25,414	745

7.6.2 Power Analysis

Simple Power Analysis (SPA). Figure 9 shows the average power consumption over 1000 traces during the initialization phase of Ascon. The $\alpha = 12$ initialization rounds (see Figure 7) are clearly identifiable. The first ∼1050 clock cycles correspond to the loading and unpacking of the shared nonce, key, and other input data transmitted from the host computer (via Python scripts) to the device. Each round of the permutation takes ∼460 clock cycles to execute.

Attack Point and First-Order T-Test Results. As shown in Figure 7, the Ascon initialization phase involves injecting the secret key, a public nonce, and a fixed initialization vector (IV) into the permutation. We target this phase for our experimental leakage evaluation.

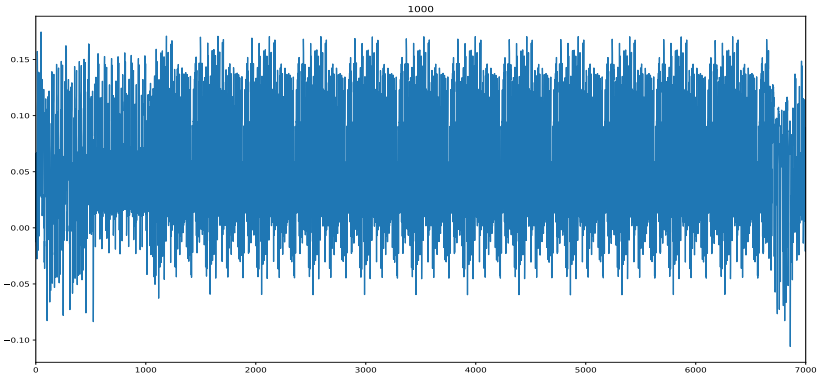


Figure 9: Average power consumption during the initialization phase of Ascon (12 rounds) using the proposed deterministic masking with $n = 2$ shares ($d = 1$ security). The x-axis denotes clock cycles; the y-axis shows power consumption.

To conduct the first-order T-test, we partition the collected power traces into two groups:

- **Group 1:** Fixed key, *fixed* nonce, fixed IV.
- **Group 2:** Fixed key, *random* nonce, fixed IV.

To understand the signal-to-noise behavior of our setup and highlight possible leakage, we first perform a sanity check by deliberately reducing the randomness. Specifically, we set the randomness used for sharing the key and nonce to zero, while preserving full randomness for the IV. Under this setting, the first few rounds are only partially protected, and the IV—being properly shared—gradually blends into the state, eventually restoring protection.

The results, depicted in Figure 10, are based on 50 power traces. The observed T-test peaks confirm that our measurement setup exhibits low noise. This is expected due to factors such as synchronized sampling and execution on a single, stable measurement platform.

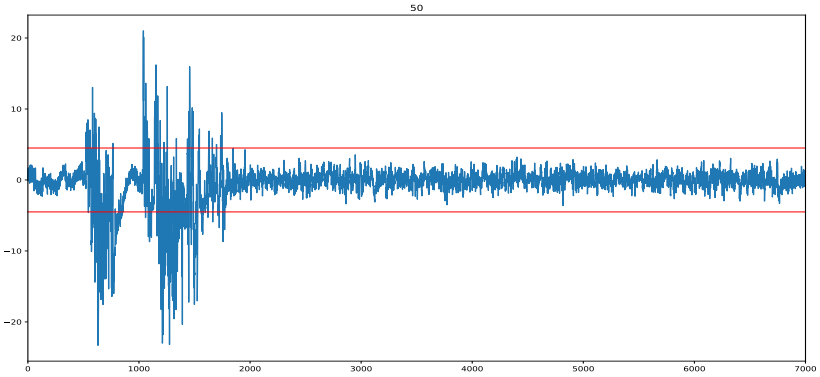


Figure 10: T-test on the initialization of Ascon with $n = 2$ shares. Randomness for sharing the key and nonce is set to zero. The x-axis denotes clock cycles and the y-axis is T-test output.

Figure 11 presents the T-test results for the $d = 1$ security case using 500K traces. All values remain within the ± 4.5 threshold, indicating no detectable leakage at this level.

However, when increasing the number of traces to 1 million, small peaks begin to appear in the early clock cycles. A closer inspection reveals that these peaks correspond to register transitions not fully mitigated by the implemented countermeasures.

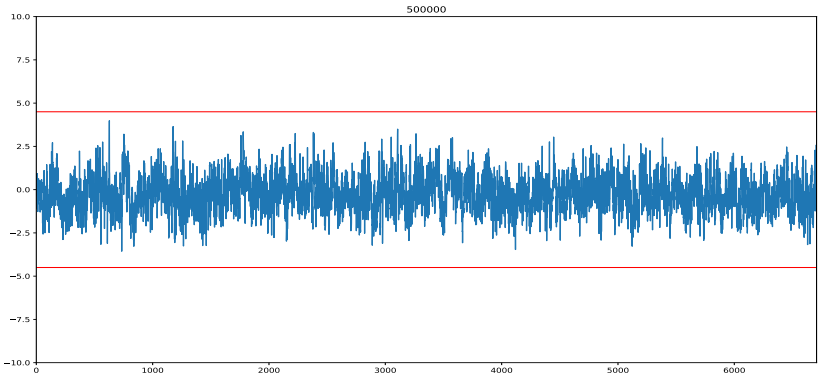


Figure 11: T-test applied to the initialization of Ascon (12 rounds) with $n = 2$ shares using the proposed deterministic masking for $d = 1$ security. Results are based on 500K traces.

In comparison, the original code from [DDM+22] exhibits noticeable peaks exceeding the ± 4.5 threshold with as few as 10K traces for the $d = 1$ case. Upon examining the final assembly code, we observed that their register-cleaning mechanism (e.g., `mov Rd, 0`) does not consistently clear registers containing secret-dependent values. However, their reported T-test results suggest that the fix is effective in suppressing leakage at this level. Similar to our implementation, their design also encounters leakage when the number of traces exceeds 1 million.

A well-known mitigation strategy for transition leakage—especially in the presence of hidden architectural registers—is to increase the masking order. In our experiments, the $d = 2$ implementation showed significantly more robustness. Even without additional transition-focused countermeasures, it did not exhibit visible leakage in our T-test measurements.

Second-Order T-test Analysis. To evaluate the effectiveness of our $d = 2$ implementation, we performed a second-order T-test targeting selected regions of the trace. Figure 12 compares the test outcomes for both the $d = 1$ and $d = 2$ cases, focusing on operations from the same round.

The heatmap on the left reveals prominent out-of-threshold spots (green) in the $d = 1$ case, indicating detectable leakage even with only 30K traces—confirming that first-order protection is insufficient against second-order attacks. In contrast, the heatmap on the right, corresponding to our $d = 2$ implementation, shows no such peaks with 1 million traces, confirming the absence of detectable second-order leakage in our setup.

8 Conclusion

This work demonstrates that higher-order probing security can be achieved through deterministic masking—relying exclusively on offline randomness embedded in the initial input shares. Our analysis reveals that the diffusion layer, characterized by the box linear branch number $\mathcal{B}_{l\text{-box}}$, serves as an implicit refresh mechanism. It effectively mixes input shares to produce seemingly fresh n-sharings in each round, without the need for additional randomness. While our practical instantiations use deterministic gadgets, our composability theorem (Theorem 1) also applies to designs that rely on online randomness inside the masked S-boxes.

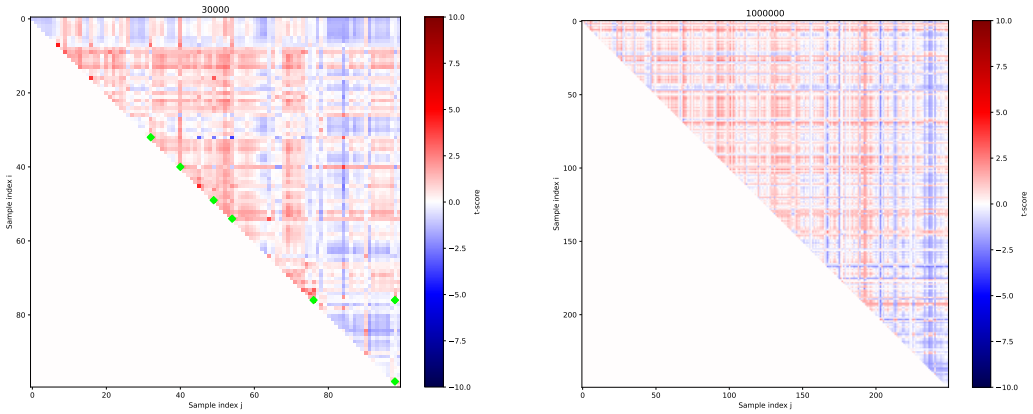


Figure 12: Second-order T-test results focused on round operations. Left: $n = 2$ shares ($d = 1$ protection), with green spots indicating values outside the ± 4.5 threshold. Right: $n = 3$ shares ($d = 2$ protection), showing no leakage. X- and Y-axes correspond to selected sample indices.

Altogether, our results pave the way for more resource-efficient masking schemes that balance strong theoretical guarantees with practical feasibility—advancing the state of countermeasures against side-channel attacks.

Acknowledgment

We would like to thank the reviewers for their valuable feedback and comments. Bart Mennink is supported by the Netherlands Organisation for Scientific Research (NWO) under grant VI.Vidi.203.099. Lejla Batina is supported by the Dutch Research Council (NWO) through the PROACT project (NWA.1215.18.014), the TTW PREDATOR project (19782), and the CiCS project of the research programme Gravitation under grant 024.006.037.

References

[BBC+19] Gilles Barthe, Sonia Belaïd, Gaëtan Cassiers, Pierre-Alain Fouque, Benjamin Grégoire, and François-Xavier Standaert. maskVerif: Automated Verification of Higher-Order Masking in Presence of Physical Defaults. In Kazue Sako, Steve A. Schneider, and Peter Y. A. Ryan, editors, *Computer Security - ESORICS 2019 - 24th European Symposium on Research in Computer Security, Luxembourg, September 23-27, 2019, Proceedings, Part I*, volume 11735 of *Lecture Notes in Computer Science*, pages 300–318. Springer, 2019.

[BBC+20] Davide Bellizia, Olivier Bronchain, Gaëtan Cassiers, Vincent Grosso, Chun Guo, Charles Momin, Olivier Pereira, Thomas Peters, and François-Xavier Standaert. Mode-Level vs. Implementation-Level Physical Security in Symmetric Cryptography A Practical Guide Through the Leakage-Resistance Jungle. In *Crypto*, Santa Barbabra, United States, August 2020.

[BBD+15a] Gilles Barthe, Sonia Belaïd, François Dupressoir, Pierre-Alain Fouque, Benjamin Grégoire, and Pierre-Yves Strub. Verified Proofs of Higher-Order Masking. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in*

Cryptology – EUROCRYPT 2015, pages 457–485, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg.

[BBD+15b] Gilles Barthe, Sonia Belaïd, François Dupressoir, Pierre-Alain Fouque, Benjamin Grégoire, Pierre-Yves Strub, and Rébecca Zucchini. Strong Non-Interference and Type-Directed Higher-Order Masking. Cryptology ePrint Archive, Paper 2015/506, 2015.

[BBD+16] Gilles Barthe, Sonia Belaïd, François Dupressoir, Pierre-Alain Fouque, Benjamin Grégoire, Pierre-Yves Strub, and Rébecca Zucchini. Strong Non-Interference and Type-Directed Higher-Order Masking. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, pages 116–129, 2016.

[BBP+16] Sonia Belaïd, Fabrice Benhamouda, Alain Passelègue, Emmanuel Prouff, Adrian Thillard, and Damien Vergnaud. Randomness Complexity of Private Circuits for Multiplication. In Marc Fischlin and Jean-Sébastien Coron, editors, *Advances in Cryptology - EUROCRYPT 2016 - 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8-12, 2016, Proceedings, Part II*, volume 9666 of *Lecture Notes in Computer Science*, pages 616–648. Springer, 2016.

[BDMS22] Tim Beyne, Siemen Dhooghe, Amir Moradi, and Aein Rezaei Shahmirzadi. Cryptanalysis of Efficient Masked Ciphers: Applications to Low Latency. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(1):679–721, 2022.

[BDPV07] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Sponge Functions. In *Ecrypt Hash Workshop 2007*, May 2007.

[BDPV13] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Keccak. In Thomas Johansson and Phong Q. Nguyen, editors, *Advances in Cryptology - EUROCRYPT 2013, 32nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Athens, Greece, May 26-30, 2013. Proceedings*, volume 7881 of *Lecture Notes in Computer Science*, pages 313–314. Springer, 2013.

[BDZ20] Tim Beyne, Siemen Dhooghe, and Zhenda Zhang. Cryptanalysis of Masked Ciphers: A Not So Random Idea. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2020 - 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7-11, 2020, Proceedings, Part I*, volume 12491 of *Lecture Notes in Computer Science*, pages 817–850. Springer, 2020.

[BGN+14] Begül Bilgin, Benedikt Gierlichs, Svetla Nikova, Ventzislav Nikov, and Vincent Rijmen. Higher-Order Threshold Implementations. In Palash Sarkar and Tetsu Iwata, editors, *Advances in Cryptology - ASIACRYPT 2014 - 20th International Conference on the Theory and Application of Cryptology and Information Security, Kaoshiung, Taiwan, R.O.C., December 7-11, 2014, Proceedings, Part II*, volume 8874 of *Lecture Notes in Computer Science*, pages 326–343. Springer, 2014.

[BNN+12] Begül Bilgin, Svetla Nikova, Ventzislav Nikov, Vincent Rijmen, and Georg Stütz. Threshold implementations of all 3 $\times$3 and 4 $\times$4 s-boxes. In Emmanuel Prouff and Patrick Schaumont, editors, *Cryptographic Hardware and Embedded Systems - CHES 2012 - 14th International Workshop, Leuven,*

Belgium, September 9-12, 2012. Proceedings, volume 7428 of *Lecture Notes in Computer Science*, pages 76–91. Springer, 2012.

[CGZ20] Jean-Sébastien Coron, Aurélien Greuet, and Rina Zeitoun. Side-Channel Masking with Pseudo-Random Generator. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020*, pages 342–375, 2020.

[CMM+24] Gaëtan Cassiers, Loïc Masure, Charles Momin, Thorben Moos, Amir Moradi, and François-Xavier Standaert. Randomness Generation for Secure Hardware Masking – Unrolled Trivium to the Rescue. *IACR Communications in Cryptology*, 1(2), 2024.

[Cor18] Jean-Sébastien Coron. Formal Verification of Side-Channel Countermeasures via Elementary Circuit Transformations. In Bart Preneel and Frederik Vercauteren, editors, *Applied Cryptography and Network Security*, pages 65–82, Cham, 2018. Springer International Publishing.

[CPRR14] Jean-Sébastien Coron, Emmanuel Prouff, Matthieu Rivain, and Thomas Roche. Higher-Order Side Channel Security and Mask Refreshing. In Shiho Moriai, editor, *Fast Software Encryption*, pages 410–424, Berlin, Heidelberg, 2014. Springer Berlin Heidelberg.

[CS20] Gaëtan Cassiers and François-Xavier Standaert. Trivially and Efficiently Composing Masked Gadgets With Probe Isolating Non-Interference. *IEEE Transactions on Information Forensics and Security*, pages 2542–2555, 2020.

[CS21] Gaëtan Cassiers and François-Xavier Standaert. Provably Secure Hardware Masking in the Transition- and Glitch-Robust Probing Model: Better Safe than Sorry. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2021(2):136–158, Feb. 2021.

[Dae95] Joan Daemen. *Cipher and Hash Function Design Strategies based on linear and differential cryptanalysis.* PhD thesis, Katholieke Universiteit Leuven, 1995.

[DBG+17] Thomas De Cnudde, Begül Bilgin, Benedikt Gierlichs, Ventzislav Nikov, Svetla Nikova, and Vincent Rijmen. Does Coupling Affect the Security of Masked Implementations? In Sylvain Guilley, editor, *Constructive Side-Channel Analysis and Secure Design - 8th International Workshop, COSADE 2017, Paris, France, April 13-14, 2017, Revised Selected Papers*, volume 10348 of *Lecture Notes in Computer Science*, pages 1–18. Springer, 2017.

[DBR+15] Thomas De Cnudde, Begül Bilgin, Oscar Reparaz, Ventzislav Nikov, and Svetla Nikova. Higher-Order Threshold Implementation of the AES S-Box. In Naofumi Homma and Marcel Medwed, editors, *Smart Card Research and Advanced Applications - 14th International Conference, CARDIS 2015, Bochum, Germany, November 4-6, 2015. Revised Selected Papers*, volume 9514 of *Lecture Notes in Computer Science*, pages 259–272. Springer, 2015.

[DDE+20] Joan Daemen, Christoph Dobraunig, Maria Eichlseder, Hannes Groß, Florian Mendel, and Robert Primas. Protecting against Statistical Ineffective Fault Attacks. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2020(3):508–543, 2020.

[DDM+22] Florian Dietrich, Christoph Dobraunig, Florian Mendel, Robert Primas, and Martin Schläffer. simpleserial-ascon: Protected ascon implementations. https://github.com/ascon/simpleserial-ascon, 2022. Accessed: July 2025.

[DEM+20] Christoph Dobraunig, Maria Eichlseder, Stefan Mangard, Florian Mendel, Bart Mennink, Robert Primas, and Thomas Unterluggauer. Isap v2.0. *IACR Trans. Symmetric Cryptol.*, 2020(S1):390–416, 2020.

[DEMS21] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schläffer. Ascon v1.2: Lightweight Authenticated Encryption and Hashing. *Journal of Cryptology*, 34, 2021.

[DHP+20] Joan Daemen, Seth Hoffert, Michaël Peeters, Gilles Van Assche, and Ronny Van Keer. Xoodyak, a lightweight cryptographic scheme. *IACR Trans. Symmetric Cryptol.*, 2020(S1):60–87, 2020.

[DNR19] Siemen Dhooghe, Svetla Nikova, and Vincent Rijmen. Threshold Implementations in the Robust Probing Model. In Begül Bilgin, Svetla Petkova-Nikova, and Vincent Rijmen, editors, *Proceedings of ACM Workshop on Theory of Implementation Security, TIS@CCS 2019, London, UK, November 11, 2019*, pages 30–37. ACM, 2019.

[DR20] Joan Daemen and Vincent Rijmen. *The Design of Rijndael - The Advanced Encryption Standard (AES), Second Edition.* Information Security and Cryptography. Springer, 2020.

[FGMDP+18] Sebastian Faust, Vincent Grosso, Santos Merino Del Pozo, Clara Paglialonga, and François-Xavier Standaert. Composable Masking Schemes in the Presence of Physical Defaults & the Robust Probing Model. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2018(3):89–120, Aug. 2018.

[FPS17] Sebastian Faust, Clara Paglialonga, and Tobias Schneider. Amortizing Randomness Complexity in Private Circuits. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology – ASIACRYPT 2017*, pages 781–810. Springer International Publishing, 2017.

[GD23a] John Gaspoz and Siemen Dhooghe. Threshold Implementations in Software: Micro-architectural Leakages in Algorithms. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2023(2):155–179, 2023.

[GD23b] John Gaspoz and Siemen Dhooghe. Threshold Implementations in Software: Micro-architectural Leakages in Algorithms. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(2):155–179, Mar. 2023.

[GMSP23] Barbara Gigerl, Florian Mendel, Martin Schläffer, and Robert Primas. Efficient Second-Order Masked Software Implementations of Ascon in Theory and Practice, NIST LWC Workshop. 2023.

[IKL+13] Yuval Ishai, Eyal Kushilevitz, Xin Li, Rafail Ostrovsky, Manoj Prabhakaran, Amit Sahai, and David Zuckerman. Robust Pseudorandom Generators. In Fedor V. Fomin, Rūsiņš Freivalds, Marta Kwiatkowska, and David Peleg, editors, *Automata, Languages, and Programming*, pages 576–588, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.

[ISW03] Yuval Ishai, Amit Sahai, and David Wagner. Private Circuits: Securing Hardware against Probing Attacks. In Dan Boneh, editor, *Advances in Cryptology - CRYPTO 2003*, pages 463–481, Berlin, Heidelberg, 2003. Springer Berlin Heidelberg.

[JPST17] Jérémy Jean, Thomas Peyrin, Siang Meng Sim, and Jade Tourteaux. Optimizing implementations of lightweight building blocks. *IACR Trans. Symmetric Cryptol.*, 2017(4):130–168, 2017.

[JSB25] Vahid Jahandideh, Jan Schoone, and Lejla Batina. Clustering approach for higher-order deterministic masking. *IACR Cryptol. ePrint Arch.*, page 273, 2025.

[KSM20] David Knichel, Pascal Sasdrich, and Amir Moradi. SILVER - Statistical Independence and Leakage Verification. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2020 - 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7-11, 2020, Proceedings, Part I*, volume 12491 of *Lecture Notes in Computer Science*, pages 787–816. Springer, 2020.

[LS16] Meicheng Liu and Siang Meng Sim. Lightweight MDS Generalized Circulant Matrices. In Thomas Peyrin, editor, *Fast Software Encryption - 23rd International Conference, FSE 2016, Bochum, Germany, March 20-23, 2016, Revised Selected Papers*, volume 9783 of *Lecture Notes in Computer Science*, pages 101–120. Springer, 2016.

[MBR19] Lauren De Meyer, Begül Bilgin, and Oscar Reparaz. Consolidating Security Notions in Hardware Masking. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2019(3):119–147, 2019.

[MMSS19] Thorben Moos, Amir Moradi, Tobias Schneider, and François-Xavier Standaert. Glitch-Resistant Masking Revisited or Why Proofs in the Robust Probing Model are Needed. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2019(2):256–292, 2019.

[MPL+11] Amir Moradi, Axel Poschmann, San Ling, Christof Paar, and Huaxiong Wang. Pushing the Limits: A Very Compact and a Threshold Implementation of AES. In Kenneth G. Paterson, editor, *Advances in Cryptology - EUROCRYPT 2011 - 30th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tallinn, Estonia, May 15-19, 2011. Proceedings*, volume 6632 of *Lecture Notes in Computer Science*, pages 69–88. Springer, 2011.

[NIS24] Ascon-Based Lightweight Cryptography Standards for Constrained Devices, (NIST initial public draft), 2024.

[NRR06] Svetla Nikova, Christian Rechberger, and Vincent Rijmen. Threshold Implementations Against Side-Channel Attacks and Glitches. In Peng Ning, Sihan Qing, and Ninghui Li, editors, *Information and Communications Security*, pages 529–545, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg.

[PAB+23] Enrico Piccione, Samuele Andreoli, Lilya Budaghyan, Claude Carlet, Siemen Dhooghe, Svetla Nikova, George Petrides, and Vincent Rijmen. An Optimal Universal Construction for the Threshold Implementation of Bijective S-Boxes. *IEEE Trans. Inf. Theory*, 69(10):6700–6710, 2023.

[RBN+15] Oscar Reparaz, Begül Bilgin, Svetla Nikova, Benedikt Gierlichs, and Ingrid Verbauwhede. Consolidating Masking Schemes. In Rosario Gennaro and Matthew Robshaw, editors, *Advances in Cryptology - CRYPTO 2015 - 35th Annual Cryptology Conference, Santa Barbara, CA, USA, August 16-20, 2015, Proceedings, Part I*, volume 9215 of *Lecture Notes in Computer Science*, pages 764–783. Springer, 2015.

[Rep15] Oscar Reparaz. A note on the security of Higher-Order Threshold Implementations. Cryptology ePrint Archive, Paper 2015/001, 2015.

[SHA15] SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions, 2015-08-04 2015.

[SM21] Aein Rezaei Shahmirzadi and Amir Moradi. Second-Order SCA Security with almost no Fresh Randomness. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2021(3):708–755, 2021.

Appendix A Proof of Lemma 5

To prove the claim, note first:

$$\begin{aligned}\mathcal{B}_l &= \min_{S\neq 0^b}\big[\mathsf{wt}(S) \;+\; \mathsf{wt}(S\mathbf{M}^\top)\big]\\ &= \min_{S\neq 0^b}\big[\mathsf{wt}\big(S \,\|\, S\mathbf{M}^\top\big)\big]\\ &= \min_{S\neq 0^b}\mathsf{wt}\Big(S\times\big[\mathbf{I}_{b\times b} \,\|\, \mathbf{M}^\top\big]\Big).\end{aligned}$$

In coding theory terms, the vector $S\times[\mathbf{I}\,\|\,\mathbf{M}^\top]$ is a *codeword* for the generator matrix $\mathbf{G}=[\mathbf{I}_{b\times b}\,\|\,\mathbf{M}^\top]$. Correspondingly, there is a *parity-check* matrix $\mathbf{H}=[\mathbf{M}\,\|\,\mathbf{I}_{b\times b}]$ such that any codeword $C\in\{0,1\}^{2b}$ satisfies

$$\mathbf{H}^\top C=\mathbf{0}_{b\times 1}.$$

Moreover, any $2b$-bit vector C with $\mathbf{H}^\top C=\mathbf{0}$ is itself a codeword. Hence,

$$\min_{S\neq 0^b}\mathsf{wt}(S\mathbf{G}) \;=\; \min_{\{C\neq 0^{2b},\,\mathbf{H}^\top C=\mathbf{0}\}}\mathsf{wt}(C).$$

After some rearrangement:

$$\min_{\{C\neq 0^{2b},\,\mathbf{H}^\top C=\mathbf{0}\}}\mathsf{wt}(C) \;=\; \min_{|\mathcal{C}|}\Big[\bigoplus_{\mathrm{col}\in\mathcal{C}}\mathbf{H}[:,\mathrm{col}]=\mathbf{0}\Big],$$

where $\mathbf{H}[:,\mathrm{col}]$ is the column vector of $\mathbf{H}$. Because $\mathbf{H}=[\mathbf{M}\,\|\,\mathbf{I}_{b\times b}]$, one can show that for $1\le l\le 2b$:

$$\mathcal{F}[2b+1-l] \;=\; [X_1,\ldots,X_b]\times\mathbf{H}[:,l].$$

Thus, for the sum of columns $\mathbf{H}[:,l]$ in $\mathcal{C}$:

$$\bigoplus_{l\in\mathcal{C}}\mathcal{F}[\,2b+1-l\,] \;=\; \big[X_1,\ldots,X_b\big]\times\Big(\bigoplus_{\mathrm{col}\,\in\,\mathcal{C}}\mathbf{H}[:,\mathrm{col}]\Big).$$

Putting it all together gives

$$\begin{aligned}\mathcal{B}_l &= \min_{|\mathcal{C}|}\Big[\bigoplus_{\text{col}\,\in\,\mathcal{C}} \mathbf{H}[:,\text{col}] \;=\; \mathbf{0}\Big] \\ &= \min_{|\mathcal{C}|}\Big[\big[X_1,\ldots,X_b\big] \times \big(\bigoplus_{\text{col}\,\in\,\mathcal{C}} \mathbf{H}[:,\text{col}]\big) = 0\Big] \\ &= \min_{|\mathcal{C}|}\Big[\bigoplus_{\text{col}\,\in\,\mathcal{C}} \mathcal{F}[\,2b+1-\text{col}\,] = 0\Big] \\ &= \min_{|\mathcal{C}'|}\Big[\bigoplus_{l\,\in\,\mathcal{C}'} \mathcal{F}[l] = 0\Big],\end{aligned} \tag{7}$$

thus completing the proof.

Remark 1. In coding theory, the value of $\mathcal{B}_l$, derived as the least number of columns of the parity check matrix that sum to zero, is an important metric. It represents the *minimum distance* of the code created with $\mathbf{G}$, which indicates the error correction capability of the code.

Appendix B Probing-Secure $\mathbb{S}\chi_5$ Gadgets

B.1 First-Order $\mathbb{S}\chi_5$ Gadget [SM21]

Let $\{A, B, C, D, E\}$ denote the five native input bits. For a first-order probing secure implementation with $n = 2$ shares, we write their corresponding shares as:

$$\{A_1, A_2, B_1, B_2, C_1, C_2, D_1, D_2, E_1, E_2\}.$$

Recall that the non-shared (unmasked) χ_5 function for five bits is:

$$\begin{aligned}&A' = A \oplus C \oplus (BC), \quad B' = B \oplus D \oplus (CD), \quad C' = C \oplus E \oplus (DE),\\ &D' = D \oplus A \oplus (EA), \quad E' = E \oplus B \oplus (AB).\end{aligned}$$

In the shared version, the output shares $\{A'_i, B'_i, C'_i, D'_i, E'_i\}$ for $i \in \{1, 2\}$ can be computed as follows:

$$\begin{aligned}A'_1 &= C_1B_1 \oplus A_1 \oplus C_1B_2 \oplus C_1, & A'_2 &= C_2B_1 \oplus A_2 \oplus C_2B_2 \oplus C_2,\\ B'_1 &= D_1C_1 \oplus B_1 \oplus D_1C_2 \oplus D_1, & B'_2 &= D_2C_2 \oplus D_2 \oplus B_2 \oplus D_2C_1,\\ C'_1 &= E_1D_2 \oplus E_1 \oplus C_1 \oplus E_1D_1, & C'_2 &= E_2D_1 \oplus E_2 \oplus C_2 \oplus E_2D_2,\\ D'_1 &= A_2E_1 \oplus D_1 \oplus A_1E_1, & D'_2 &= A_1E_2 \oplus D_2 \oplus A_1 \oplus A_2E_2 \oplus A_2,\\ E'_1 &= B_1A_1 \oplus E_1 \oplus B_1 \oplus B_1A_2, & E'_2 &= B_2A_1 \oplus E_2 \oplus B_2 \oplus B_2A_2.\end{aligned}$$

B.2 Second-Order $\mathbb{S}\chi_5$ Gadget [SM21]

For second-order probing secure implementations ($n = 3$ shares) of χ_5 function, let the set $\{A_1, A_2, A_3, B_1, B_2, B_3, \ldots, E_1, E_2, E_3\}$ denote the three shares of inputs $\{A, B, C, D, E\}$. The output shares of the χ_5 function can be computed as:

$$
\begin{aligned}
A'_1 &= C_1B_1 \oplus C_2 \oplus C_2B_1 \oplus C_3 \oplus A_1 \oplus C_3B_1,\\
A'_2 &= C_1 \oplus C_1B_2 \oplus C_2B_2 \oplus C_3 \oplus A_2 \oplus C_3B_2,\\
A'_3 &= A_3 \oplus C_1B_3 \oplus C_2B_3 \oplus C_3 \oplus C_3B_3,
\end{aligned}
$$

$$
\begin{aligned}
B'_1 &= D_1 \oplus D_1C_1 \oplus D_2C_1 \oplus B_3 \oplus D_3C_1,\\
B'_2 &= D_1 \oplus D_1C_2 \oplus D_2C_2 \oplus D_3 \oplus B_1 \oplus D_3C_2,\\
B'_3 &= D_1 \oplus B_2 \oplus D_1C_3 \oplus D_2 \oplus D_2C_3 \oplus D_3C_3,
\end{aligned}
$$

$$
\begin{aligned}
C'_1 &= E_1 \oplus C_3 \oplus E_1D_1 \oplus E_2D_1 \oplus E_3 \oplus E_3D_1,\\
C'_2 &= E_1 \oplus E_1D_2 \oplus E_2D_2 \oplus C_1 \oplus E_3D_2,\\
C'_3 &= E_1 \oplus C_2 \oplus E_1D_3 \oplus E_2 \oplus E_2D_3 \oplus E_3D_3,
\end{aligned}
$$

$$
\begin{aligned}
D'_1 &= A_1 \oplus D_1 \oplus A_1E_1 \oplus A_2 \oplus A_2E_1 \oplus A_3E_1,\\
D'_2 &= A_1 \oplus A_1E_2 \oplus A_2E_2 \oplus A_3 \oplus D_3 \oplus A_3E_2,\\
D'_3 &= A_1 \oplus A_1E_3 \oplus A_2E_3 \oplus D_2 \oplus A_3E_3,
\end{aligned}
$$

$$
\begin{aligned}
E'_1 &= B_1 \oplus E_1 \oplus B_1A_1 \oplus B_2 \oplus B_2A_1 \oplus B_3A_1,\\
E'_2 &= B_1 \oplus B_1A_2 \oplus B_2A_2 \oplus E_3 \oplus B_3A_2,\\
E'_3 &= B_1 \oplus B_1A_3 \oplus B_2A_3 \oplus B_3 \oplus E_2 \oplus B_3A_3.
\end{aligned}
$$

Important note: The order of operations (from left to right) is critical for achieving probing security. The original threshold implementations in [SM21] ensures security even in the presence of glitches, while the simplified expressions given here are probing-secure and suffice for scenarios considered in our work.